UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Duong Minh Quang – Le Quoc Van

cho font size nhỏ lại để không bị rớt thành 3 hàng

INNOVATING VIDEO CONFERENCING SYSTEMS THROUGH BLOCKCHAIN

BACHELOR THESIS IN INFORMATION TECHNOLOGY
ADVANCED PROGRAM IN COMPUTER SCIENCE

Ho Chi Minh City, July 2026

UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Duong Minh Quang – 22125081
Le Quoc Van – 22125119

INNOVATING VIDEO CONFERENCING SYSTEMS THROUGH BLOCKCHAIN

BACHELOR THESIS IN INFORMATION TECHNOLOGY
ADVANCED PROGRAM IN COMPUTER SCIENCE

THESIS SUPERVISOR
Assoc. Prof. Nguyen Dinh Thuc

Ho Chi Minh City, July 2026

Declaration

We hereby declare that this thesis is our own research work, carried out under the supervision of Assoc. Prof. Nguyen Dinh Thuc.

Thesis Revision Statement

The thesis revision statement (required if the thesis title or report content changed after the defence) is inserted here. It is prepared for the bound copy after the defence session.

Supervisor's Assessment

The signed assessment sheet of the thesis supervisor, provided by the Academic Affairs Office, is inserted here.

Reviewer's Assessment

The signed assessment sheet of the thesis reviewer, provided by the Academic Affairs Office, is inserted here.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Assoc. Prof. Nguyen Dinh Thuc, for his guidance and constructive feedback throughout the design, implementation, evaluation, and preparation of this thesis.

We thank the Faculty of Information Technology and the Advanced Program in Computer Science at the University of Science, VNU-HCM, for the coursework, facilities, and academic environment that supported this work.

Finally, we thank our families and friends for their support and patience throughout this work.

Contents

List of Figures

List of Tables

This thesis studies solutions and proposes a decentralized video conferencing (DVConf) system to fill this gap.

Abstract

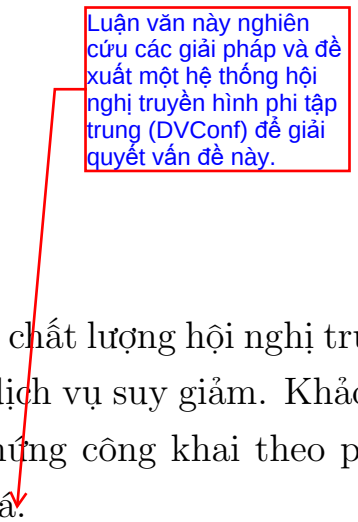
Real-time video conferencing remains operator-run: one provider controls admission, policy, and quality decisions, leaving users little independent proof of degradation. Surveyed decentralised systems lack public per-session evidence linked to tested real-time media.

DVConf keeps media off-chain and anchors signed QoS evidence plus accounting events on Sui. It combines Move contracts, services, forensic tools, and a browser client. Contracts record obligation and reputation effects; deduction is a later relay-supplied transaction against its stake, not automatic seizure. Settlement averages two validator proofs; the three-of-four rule is a design bound. Room, TURN, and relay admission stay separate. SFrame may fail open when attachment fails.

Measured outcomes are reported with their boundaries. One-way media latency on a Malaysia laptop–relay path was $p_{95} = 70.9$ ms — a lower-bound component sum assembled from half-RTTs, not a capture-to-paint measurement of the sub-200 ms target. On matched Azure inter-region paths, enabling end-to-end encryption added a +1.86 ms mean one-way overhead ($n = 30$ per arm; 95% CI $[+1.14, +2.60]$ ms; tail difference unresolved; consumer access paths unmeasured). Forwarding scaled linearly to 15 viewers; 100 remains projected. A pinned-localnet $K = 2$, $N = 4$ session cost about 0.035 SUI net (0.017 SUI irreversible); mainnet cost is unmeasured. Off-chain services observed the room-creation and settlement events about five seconds after the SDK call (p_{50} 5.0/4.7 s; $n = 30$ each), a figure that includes the production poller's 5 s interval and warm,

non-independent state; it is not mainnet finality or browser-visible join/recovery latency. In-process warm-pipe cutover reached $p_{95} = 74$ ms, and standby promotion was observed on-chain, but rendered continuity and full recovery time stay unmeasured.

Tóm tắt



Nhà cung cấp kiểm soát truy cập, chính sách, chất lượng hội nghị truyền hình; người dùng thiếu bằng chứng độc lập khi dịch vụ suy giảm. Khảo sát chưa thấy giải pháp phi tập trung gắn bằng chứng công khai theo phiên với đa phương tiện thời gian thực được đánh giá.

DVConf giữ đa phương tiện ngoài chuỗi, gồm hợp đồng Move, dịch vụ, điều tra và web, và neo bằng chứng QoS đã ký cùng sự kiện kế toán trên Sui. Hợp đồng ghi nghĩa vụ phạt và danh tiếng; khấu trừ cần relay tự dùng khoản đặt cọc, không tự động tịch thu. Quyết toán trung bình hai chứng thực; 3/4 là cận thiết kế. Phòng, TURN và tiếp nhận relay tách biệt; SFrame có thể dùng bản rõ khi lỗi gắn bộ biến đổi.

Độ trễ một chiều tuyến laptop–relay Malaysia đạt $p_{95} = 70,9$ ms — cận dưới ghép từ nửa RTT, chưa phải thu hình–hiển thị dưới 200 ms. Trên tuyến Azure liên vùng đồng nhất, mã hóa đầu-cuối thêm trung bình $+1,86$ ms một chiều ($n = 30$/nhánh; CI 95% $[+1,14; +2,60]$ ms; đuôi chưa phân giải; tuyến người dùng chưa đo). Chuyển tiếp tuyến tính tới 15 người xem; 100 là dự phóng. Phiên $K = 2$, $N = 4$ trên localnet ghim tốn khoảng 0,035 SUI ròng (0,017 không hoàn lại); mainnet chưa đo. Dịch vụ ngoài chuỗi thấy sự kiện tạo phòng, quyết toán khoảng năm giây sau lời gọi SDK (p_{50} 5,0/4,7 s; $n = 30$/loại), gồm chu kỳ quét 5 s và trạng thái warm không độc lập; chưa phải finality mainnet hay trễ tham gia/khôi phục trên trình duyệt. Warm-pipe cùng tiến trình đạt $p_{95} = 74$ ms; thăng cấp dự phòng đã thấy trên chuỗi, nhưng liên tục hiển thị và tổng thời gian khôi phục chưa đo.

Chapter 1

Introduction and Motivation

1.1 Motivation

Contemporary real-time video conferencing is commonly delivered through operator-controlled services. The representative systems examined in Chapter 2 — Zoom, Google Meet, and operator-deployed LiveKit-style SFUs — share a structural property in their public specifications: the operator controls admission, the assessment of misbehaviour, and the applicable policy. Subsequent mechanisms for authorisation, penalties, and failover therefore either retain that authority or assign it to a different trust root.

The survey identifies three recurring trust assumptions. **Admission authority**: LiveKit's `VideoGrant` is signed by an operator-held API secret, while coturn's TURN credentials derive from an operator-held shared secret; these credential mechanisms do not themselves provide an externally verifiable appeal path. **Assessment authority**: the same operator supplies the detector and determines the consequence, without a public record of the detector's observations. **Policy authority**: terms, regional policy, recording defaults, and pricing remain operator-mutable between sessions. Although each choice is operationally plausible, their combination concentrates session authority in one party. DVConf therefore asks whether the *trust root* can be relocated without sacrificing the latency

(mở ngoặc ghi đầy đủ từ tiếng Anh)

and reliability required for interactive conferencing; it does not claim that centralised RTC is inherently unsafe.

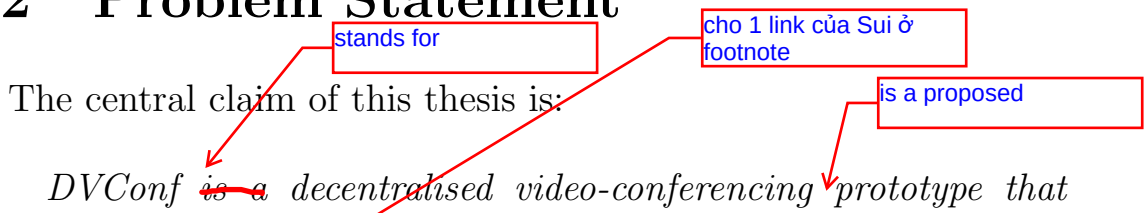
Existing decentralised RTC provides only a partial answer. Chapter 2 (§2.3) examines three documented systems against the centralised endpoints from which they depart. **Jami** [1], [2] removes the operator but supplies no public quality or settlement record. **Huddle01** [3], [4] anchors economic settlement while signalling remains operator-controlled. **Datagram** [5] moves authority to a validator committee whose internal operation is not publicly specified. The trust-root matrix in Chapter 2 (Table 2.2) therefore motivates DVConf's narrower position: signed per-session QoS attestations and resulting economic decisions on a publicly inspectable event history. This makes signatures and decisions verifiable, but it does not establish that every attested metric is correct: part of what the measuring validators attest is data the relay reports about itself, and the on-chain proof check does not yet confirm that the relay named in a proof was the one assigned to the room (§6.3).

state of the arts (SOTA)

The thesis evaluates two properties the ~~related-work chapter~~ cannot supply. First, **chain settlement is excluded from the steady-state media-forwarding path**: Sui-anchored assignment, obligation recording, and settlement are partitioned from the evaluated mediasoup [6] SFU path, with the ffmpeg MCU as an optional plaintext-only mode. On a pinned localnet — a local Sui network pinned to one protocol version — the services that act on chain events saw them about five seconds after transaction submission, a figure that includes the event poller's 5 s polling interval; the measurement runs from the SDK submit call to delivery of the exact `RoomCreated` and `RewardsDistributed` events through the unchanged production event poller (§5.2). That number is deliberately narrow in scope: it is not a measurement of finality on the public mainnet, of the time a participant waits to join a room, or of full recovery from a killed process to media rendered on screen (§6.4). Second, **the evidence-and-obligation boundary is explicit**: two distinct validator proofs can

drive rewards, while the zero-quality settlement branch records an obligation, sets reputation to zero, and emits `RelaySlashed`. Penalties are thus recorded on-chain but collected cooperatively: the deduction happens only when the penalised relay itself later submits the paying transaction (`pay_slash`) against its own stake, because that stake is not locked for the room lifetime (§3.4). The partly relay-self-reported probe data and the missing assigned-relay check noted above bound the claim in the same way. The prototype therefore demonstrates signed evidence and an on-chain consequence workflow, not permissionless bond seizure or an enforced three-of-four Byzantine quorum (§6.3).

1.2 Problem Statement



The central claim of this thesis is:

> *DVConf ~~is a~~ decentralised video-conferencing prototype that keeps steady-state media off-chain while anchoring signed per-session quality-of-service attestations and economic decisions on Sui; the implementation demonstrates the workflow, while complete proof-to-relay binding and permissionless bond seizure remain open.*

This is a research claim rather than a product claim; it does not assert performance superior to Zoom, enterprise-scale capacity, or completed permissionless slashing. It asks how far a chain-anchored control and economic plane can be integrated with an off-chain RTC media plane while exposing the remaining trust boundaries.

The approved proposal frames four persistent research gaps. **(1) Resource utilisation**: surplus edge and IoT computing resources lack a participation mechanism. **(2) Real-time performance**: geographically distributed volunteer relays must be evaluated against a sub-200 ms target. **(3) Incentives and QoS**: detection needs a tamper-evident artefact and

a credible consequence. **(4) System integration**: chain settlement and real-time media require an explicit boundary. The thesis addresses all four at materially different evidence depth.

Evidence is most complete for **gaps 3 and 4**, at prototype depth. The project's Architecture Decision Records (ADRs) document implementation rationale; they are traceability artefacts rather than independent scholarly evidence. These records specify stake and reward policy, a three-class taxonomy for proposed penalties, and a design with four validators and a three-of-four threshold. The implementation is narrower than this design in known, explicitly reported ways: settlement requires two distinct proofs rather than three of four; its two-value "median" is an arithmetic mean; and the zero-quality settlement branch records an obligation, but collection requires a later transaction from the relay itself (`pay_slash`) and its bond is not locked for the room lifetime. The integration contribution is a working three-repository boundary (§4.1); Chapter 4 explicitly records the interfaces that remain incomplete, including gaps in event coverage, duplicated decoding logic, and a relay-admission check that is not yet enforced.

Evidence is more limited for **gaps 1 and 2**. The relay registry permits public applications but gates admission by stake and voting; no IoT client is demonstrated. The scope-triage entry D2 places IoT participation in future work, and Chapter 6 (§6.2) specifies the scope reductions an IoT-class relay would require. Measured one-way latency on the evaluated wide-area-network (WAN) path came in well under the 200 ms target, at $p_{95} = 70.9$ ms — but as a lower bound: the figure is a reduced-fidelity *component sum* derived by halving round-trip times, not a capture-to-paint measurement, so the full user-perceived latency would be higher. Validation with a real camera and two distinct Internet service providers therefore remains future work. The evaluation ran between a laptop on a residential-ISP connection and one Azure relay VM in Malaysia ($n = 30$ sessions; full percentiles in §5.2).

Two numerical claims retain explicit dispositions. Sub-200 ms is a de-

sign target supported only by the lower-bound run above. The proposal's approximately USD 0.01 figure is a preliminary estimate for room creation, not a complete-session acceptance threshold. A full session executed on the pinned localnet — two validators scoring four relays ($K = 2$, $N = 4$) — cost about 0.035 SUI net, of which about 0.017 SUI is irreversible: a few US cents at the labelled USD 1.50/SUI rate. Section 5.3 reports the measured per-function localnet gas alongside the exact figures and the 13-transaction breakdown. Absolute mainnet cost remains unmeasured, and this cost result does not validate the designed three-of-four settlement rule.

1.3 Contributions

The thesis groups its contribution into three claims rather than treating every component or methodology document as a distinct novelty claim.

Chain-anchored RTC control and economics. In user terms, the integration means a participant can create a room, join it, and exchange live audio and video while admission, relay assignment, and reward settlement are decided and recorded on a public chain rather than in an operator's database. To that end DVConf integrates stake-gated registries, control-plane pairing proposals, a quorum-issued room authorisation token (`RoomCapability`) enforced at the standalone signalling service, separate TURN-HMAC issuance, reward settlement, and a React client with an off-chain mediasoup plane. A five-member control-plane localnet run exercised four-of-five score-group finalisation, while the evaluated default uses one control plane and WAN multi-host operation remains open. The contribution is the integrated boundary, not complete decentralisation: known governance and enforcement gaps remain, and §6.2 reports them. In compressed form, when several pairing proposals tie on score the first proposal's assignment vector is used, so same-score proposals with different assignments can still form a quorum; relay admission does not yet check the room authorisation token; TURN credentials come from a separate mechanism

under a single control-plane capability (`ControlPlaneCap`) with no operational coturn deployment behind it; and a substantial set of operations remains under the deployer's administrative capability (`AdminCap`).

Accountable media-operation evidence. The second contribution is that relay behaviour during a session leaves a signed, publicly inspectable evidence trail: validator daemons produce dual-signed `SessionProof` attestations, and the forensic CLI summarises the covered proof, obligation, relay-history, and reward events without recording media. The measurement behind each attestation is a composite probe: the validator measures RTT, jitter, and loss directly over STUN, while forwarded bytes, peer count, and duration come from the relay's own HTTP endpoint; the record therefore authenticates who attested but does not independently verify the relay-reported counters or the participant media path. Enforcement is narrower than the design in disclosed ways: settlement accepts two proofs rather than the designed three-of-four threshold; accepting a proof does not check that the relay was assigned to the room; and `RelaySlashed` records an obligation whose actual deduction happens only if the relay itself later submits a `pay_slash` transaction against its own stake (`StakePosition`), which is currently not locked. The contribution is therefore a signed evidence-and-obligation workflow with disclosed binding and custody gaps, not permissionless deduction.

Reproducible evaluation. The latency, cost, capacity, failover, end-to-end encryption (E2EE), and forensic harnesses keep evidence classes explicit. Measured WAN latency came in under the sub-200 ms target on the evaluated path, at $p_{95} = 70.9$ ms — a reduced-fidelity lower-bound component sum rather than capture-to-paint (§5.2). A full session with two validators scoring four relays ($K = 2$, $N = 4$) was executed and its cost measured directly on the pinned localnet, covering eight proof submissions plus five fixed lifecycle calls (§5.3). Media capacity is measured through $N \leq 15$, with the $N = 100$ figure a projection rather than a measurement (§5.4). Relay failover rests on an in-process warm-pipe — a pre-

established standby media pipe inside one process — whose cutover floor is $p_{95} = 74$ ms over 30 cutovers (full percentiles in §5.5); this mechanism floor is distinct from the failure-injection experiment, which establishes media delivery before relay termination and subsequent on-chain promotion but not browser-rendered media after promotion or full mean time to recovery (MTTR). On matched Azure inter-region cloud-WAN paths, enabling E2EE added a +1.86 ms mean one-way overhead ($n = 30$ per arm; 95% CI $[+1.14, +2.60]$ ms); the tail (p_{95}) difference was unresolved, and consumer-access-network paths remain unmeasured (§5.2). The methodology documents support the reproducibility of this contribution.

The bounded-novelty statement claims no novelty for slashing (Sarmenta [7]; Tendermint [8]), PBFT's $n \geq 3f + 1$ bound [9], scoped grants, coturn HMAC, mediasoup forwarding, or DePIN framing [3], [4]. Within the systems and sources surveyed in Chapter 2, the narrow claim is their **combination** in a public-chain RTC control and economic plane with off-chain media and per-session signed attestations. The RTC quality-fault class documented in ADR-0003 is a **design extension**; full enforcement of that signalling policy, three-of-four settlement, complete relay binding, and protocol-enforced bond deduction are not implemented.

1.4 Thesis Structure

The remainder of the manuscript comprises six chapters followed by the references. Each chapter is mapped to a motivation, research gap, or contribution introduced in §1.1–§1.3.

Chapter 2 (Background and Related Work) reviews WebRTC, SFU and MCU topologies, production RTC trust roots, the accountability lineage, and the surveyed Sui Move–WebRTC gap. It supports the prior-art and bounded-novelty claims.

Chapter 3 (Proposed Architecture) defines the goals, system layers, component boundaries, trust model, session lifecycle, capability mech-

anisms, and coordination design. **Chapter 4 (Implementation)** maps those decisions to the three repositories and records incomplete integration seams.

Chapter 5 (Evaluation) reports the methodology and the latency, cost, capacity, failover, and validity analyses. It distinguishes measurements from projections and design targets. **Chapter 6 (Discussion and Limitations)** compares the implementation with the proposal, groups remaining work, re-derives the trust bounds, and states Sui-specific caveats.

Chapter 7 (Conclusion and Future Work) relates the three contribution groups to the central claim and four research gaps, then prioritises the remaining evidence and implementation work. The references use IEEE numbering by first citation.

Chapter 2

Background and Related Work

This chapter is organised around two questions: *what does real-time video conferencing technically require* (§2.1–§2.2), and *who has tried to decentralise it, with what trust roots* (§2.3–§2.5). The first two sections define the protocol substrate and latency budget; the remaining three map the surveyed production landscape and delimit the gap addressed by this thesis.

2.1 WebRTC Primer and Lineage

2.1.1 The WebRTC Protocol Stack

Modern in-browser real-time communication rests on three IETF building blocks:

- **ICE.** Interactive Connectivity Establishment is specified by RFC 5245 [10], which RFC 8445 [11] obsoleted in 2018 but which is cited here for consistency with the thesis proposal. ICE composes host candidates, server-reflexive candidates from STUN, and relayed candidates from TURN into a single candidate-gathering and connectivity-check protocol. The relayed path is the one DVConf inherits from `coturn` [12] when symmetric-NAT clients cannot reach each other directly (§3.6).

- **Transport Layer Security 1.3** — RFC 8446 [13]. Protects the WebSocket signalling channel (WSS). In DVConf this is a deployment-layer property: the signalling daemon itself binds plain WebSocket, and TLS terminates at a fronting reverse proxy — the configuration used in the wide-area-network (WAN) evaluations reported in Chapter 5 — with in-daemon WSS enforcement an acknowledged hardening item. The media channel is keyed separately: per the WebRTC security architecture [14], each peer connection runs a DTLS handshake whose exported keys drive SRTP (DTLS-SRTP [15], with DTLS 1.2 as the mandated baseline). DTLS is TLS's datagram sibling — a distinct protocol, not TLS 1.3 tunnelled over UDP.

- **SRTP.** The Secure Real-time Transport Protocol, specified by RFC 3711 [16], encrypts and authenticates RTP audio and video frames between the client and the relay after DTLS-SRTP key agreement.

DVConf claims no protocol-level innovation at this layer. The browser's native WebRTC API and the relay-side `mediasoup` library [6] implement the media-plane RFCs (ICE, DTLS-SRTP, SRTP) unchanged; the contribution lies above the SRTP boundary, in signalling orchestration and per-session evidence.

2.1.2 Why Latency Is the Binding Constraint

The protocols above were designed for sub-second interactive use. ITU-T Recommendation G.114 [17] places one-way mouth-to-ear delays up to 150 ms in the "good" range and gives 400 ms as a general planning limit. This thesis adopts **sub-200 ms glass-to-glass latency** as an engineering design ceiling, which lies between those reference points. The reduced-fidelity lower-bound component-sum estimate reported in §5.2 is below

150 ms on the tested path, but direct capture-to-paint validation remains open.

The latency budget motivates DVConf's separation of media from chain coordination. Sui's platform-level consensus commit is sub-second, while its block and checkpoint cadence is seconds-scale [18]. DVConf separately measures a pinned single-host localnet path from immediately before the SDK submit call to exact matched-event delivery by the unchanged production poller at a 5 s interval (§5.2). This polling-inclusive application observation is neither a measurement of platform or mainnet finality nor a join, revocation, or failover measurement. DVConf therefore excludes chain interaction from the steady-state per-frame media path by design. This separation is also used by the hybrid systems surveyed in §2.3, where the chain anchors *state* while media stays off-chain; it recurs in §3.7 and §5.2.

2.1.3 The Federated–Consolidated–Federated Arc

The 2020s "decentralised infrastructure" wave is the third phase of a longer cycle. Foster, Kesselman, and Tuecke's grid-computing paper [19] established Virtual Organizations as the unit of cross-domain resource sharing in 2001. Buyya, Abramson, and Giddy [20] then surveyed auction, commodity-market, and posted-price models for grid scheduling; this economic vocabulary also appears in later DePIN reward designs, including DVConf's scarcity-ratio settlement (§3.7). The early 2010s consolidated the federated tradition into single-provider elastic cloud [21], [22], [23], with pay-per-use becoming the dominant deployment model.

The wave considered here is a second phase of federation, using cryptographic-economic mechanisms alongside administrative or contractual trust. Public blockchains provide a substrate on which DVConf can test session-scale anchoring; this design choice does not establish acceptable application latency. The bounded localnet submit-to-event result in

§5.2 includes a 5 s polling cadence and does not establish public-network, browser, join, or recovery latency.

2.2 SFU versus MCU Paradigms

Multi-party RTC has three structural topologies, distinguished by *where* the per-stream fan-out work runs. Each trades client cost, server cost, and trust assumptions differently.

2.2.1 Full Mesh: Quadratic Scaling

In a pure peer-to-peer mesh of N participants, every pair maintains its own bidirectional SRTP flow: connection count grows as $N(N-1)/2$ and per-client upload grows as $N-1$. At $N = 5$ a typical webcam stream already costs roughly four times the bandwidth of a one-to-one call, with encoder fan-out scaling similarly. Full mesh is therefore practical only for small calls. DVConf does not implement full mesh as a primary topology; the participant-scaling behaviour measured in §5.4 is a relay- and consumer-side effect, not a mesh ceiling.

2.2.2 SFU: Selective Forwarding from a Relay

The Selective Forwarding Unit topology replaces the quadratic mesh with a 1:N star centred on a relay. Each client uploads one stream; the relay forwards it to the other $N-1$ participants without transcoding. Per-client upload becomes constant, while aggregate egress concentrates at the relay. Two reference systems shape DVConf's SFU design:

- **mediasoup** [6] (ISC-licensed; Node.js bindings over C++ media workers) implements SFU semantics through a `Router`, `Transport`, `Producer`, and `Consumer` API with simulcast and SVC support. DVConf retains and extends its `PipeTransport` and `pipeToRouter`

primitives for cross-worker and cross-host stream mirroring. Project-specific ADR-0004 records this implementation choice; it is a traceability artefact rather than independent scholarly evidence. §3.5 describes the implementation boundary.

- **LiveKit** [24] (Apache-2.0; Go) is an open-source production SFU built around a `VideoGrant` capability token and Redis-routed reconnection. Its scoped grant is a conceptual precedent for DVConf's `RoomCapability`; TURN HMAC issuance is a separate mechanism (§3.6). LiveKit's issuer is an operator holding an API secret, whereas DVConf's room capability is issued through a chain-anchored control-plane path.

The SFU is a common production topology among the surveyed systems and is DVConf's default topology.

2.2.3 MCU: Server-Side Composite Mixing

The older Multipoint Control Unit topology has every client upload to a server that *decodes, composites, and re-encodes* one mixed stream for all participants. Per-client download drops to a constant (versus $N - 1$ for SFU) at the price of the heaviest CPU pipeline in real-time media running server-side. MCU suits sessions where downstream bandwidth binds — cellular participants, or recordings consumed by audiences larger than the speaker set.

DVConf adopts the SFU as its default and evaluated media plane for two design reasons:

1. **End-to-end encryption rules the MCU out when enabled.** An MCU must decode, composite, and re-encode plaintext media. DVConf offers an opt-in SFrame-style envelope above SRTP [25]; when the browser transform attaches successfully, the relay forwards

opaque payloads without a content key. Transform attachment can fail open to plaintext, so this is not a universal property of every room. A compositing relay is incompatible with the encrypted mode.

2. **Simulcast provides downstream adaptation without decoding.** The downstream-adaptation benefit of an MCU is obtained instead by having each sender publish a three-layer simulcast ladder (a native mediasoup capability) and each receiver subscribe, per tile, to the layer it actually renders: low bitrate for thumbnails, full quality for the pinned speaker, and no subscription for off-screen tiles. Downstream cost then scales with what the viewer displays, while the relay remains forward-only.

A relay-side composite path (`ffmpeg` `xstack` as a child process of the relay daemon) exists in the implementation. The original composite-first rationale recorded in ADR-0004 was superseded by the subsequent end-to-end-encryption decision (§3.4). This plaintext-only capability lies outside the end-to-end-encrypted envelope: an encrypted room refuses the mixer, and dedicated evaluation of the composite path remains future work (§6.2).

2.2.4 P2P-MCU: Client-Side Composite

A third topology pushes the MCU role into one of the participating browsers. Ng et al. [26] elect one client as the composite mixer: it receives the other streams, composes a picture-in-picture, and re-encodes one downstream stream. Their measurements report a **64% CPU reduction and 35% bandwidth reduction at the non-MCU peers** relative to full mesh, with the cost concentrated at the elected client. DVConf adopts the linear-star structure but relocates the central role to a forward-only relay. Table 2.1 summarises the trust-model difference.

The principal inherited numerical result is the reduction from $N(N-1)/2$ connections to a linear star. The 64% CPU reduction and 35%

Table 2.1: P2P-MCU and DVConf: similar connection reduction, different trust models.

Concept	**P2P-MCU [26]**	**DVConf**
Central node	Elected browser client	Staked relay daemon with signed evidence
Work at the centre	Decode, composite, and re-encode plaintext	Forward-only; content-blind when end-to-end encryption is enabled (§2.2.3)
Client burden	High at the elected client	Symmetric; per-layer simulcast subscription
Primary failure response	Re-elect after the selected client leaves	Promote a standby relay after heartbeat staleness (ADR-0004)
Accountability	No protocol consequence	`SessionProof`, recorded obligation, and later cooperative deduction

bandwidth reduction are P2P-MCU measurements and remain attributed to [26]. DVConf's efficiency figures come from its own evaluation (§5.2, §5.3).

2.2.5 DVConf's Position on the Topology Axis

DVConf selects the SFU topology for two design reasons: end-to-end encryption requires a forward-only relay, and simulcast supplies the downstream adaptation that an MCU would otherwise provide (§2.2.3). Room-to-relay assignment occurs once per room through the control-plane agreement path (§3.5); continuous per-packet load balancing is outside the implemented scope (§3.3). The failover and capacity evaluations are therefore scoped to the SFU path (§5.4, §5.5), which is the system's evaluated mode.

2.3 Selected RTC Systems

The protocol stack of §2.1 and the topology axis of §2.2 are neutral about *who runs the relays, who issues credentials, and who can audit the system*. The surveyed systems answer those questions differently. This section organises the design space along a *trust-root* axis — the authority for session state — and compares five trust-root categories (Table 2.2), with Zoom, Google Meet, and hosted LiveKit grouped in one centralised category. The term decentralised physical infrastructure network (DePIN) describes the decentralised part of this matrix, and DVConf adopts that framing.

Table 2.2: Trust roots across the selected RTC systems.

System	**Token and state layer**	**Signalling and coordination**	**Media path**	**Evidence posture**
Zoom, Google Meet, hosted LiveKit	operator-controlled tokens and state	centralised	centralised SFU	operator-controlled service or deployment
Jami	none (P2P)	OpenDHT distributed hash table	full mesh	open source; no public per-session evidence identified
Huddle01	Arbitrum Orbit Layer 3 and HUDL token	off-chain	DePIN media-node mesh	open hybrid; node entry gated by a non-fungible token
Datagram	Avalanche Layer 1	reported as on-chain	closed DePIN	commercial, closed, and patented
DVConf	Sui	off-chain WebRTC with chain-anchored proofs	forward-only SFU	public research prototype; signed per-session evidence

2.3.1 Centralised Software as a Service

The mainstream systems concentrate trust in a single operator: Zoom and Google Meet are proprietary services, and many LiveKit deployments use a hosted control plane with operator-issued JWT credentials [24]. This is the status quo from which DVConf departs. DVConf implements software paths for chain-anchored room capabilities and TURN credential delivery, but it does not demonstrate an operational coturn deployment (§3.6).

2.3.2 Pure P2P: Jami

At the opposite extreme, Jami [1] runs no central media server: peer discovery uses OpenDHT [2], a Kademlia-style distributed hash table; NAT traversal retains auxiliary infrastructure; identity is key-based; and end-to-end encryption is the default. Multi-party calls inherit the full-mesh scaling of §2.2.1. Within the public mechanisms surveyed here, Jami has no public per-session evidence-and-consequence surface of the kind examined in this thesis; misbehaviour is handled locally by peers.

2.3.3 Decentralised Hybrid: Huddle01

Huddle01 [3], [4] provides a production analogue in the surveyed corpus. Its published architecture (retrieved May 2026) uses an Arbitrum Orbit L3 with AnyTrust [27], [28] for registration, staking, and rewards, while media nodes and signalling remain off-chain. Its published mechanism uses reputation pressure and does not describe cryptographically signed per-session evidence as the basis for protocol deduction. The cited report [3] describes the "Guardian Pass" non-fungible-token node sale as the bootstrap mechanism, so node entry is token-gated rather than based on DVConf's stake-gated registry. DVConf shares the DePIN category and chain-anchored registry, but adds `SessionProof` records, an RTC-specific

proposed penalty taxonomy (ADR-0003), and forensic reports for selected proof, obligation, relay-history, and reward events. These reports do not reconstruct the complete session or consensus history.

2.3.4 Decentralised Closed: Datagram

Datagram [5], [29] occupies a commercial decentralised position in the matrix: an Avalanche L1 subnet chain and a "Hyper Network Layer" whose coordination mechanism is not publicly specified in sufficient detail for this comparison. Its public materials do not expose the consensus, penalty rules, or evidence shape at mechanism level. The comparison is therefore limited to documented architecture and public auditability, not vendor scale claims.

2.3.5 Unrepresented Configuration

The configuration combining *decentralised state, decentralised per-session evidence, and a publicly available research implementation* is not represented among the production systems surveyed in Table 2.2. DVConf examines this configuration through four design choices whose combination was not identified in the survey:

1. **Sui** [18] as the public trust root for per-session coordination — rather than an Arbitrum-stack L3 (Huddle01) or Avalanche L1 (Datagram); the localnet submit→event result in §5.2 is polling-inclusive and not mainnet finality;

2. **hybrid anchoring**: off-chain WebRTC signalling paired with on-chain `SessionProof` records; unlike the surveyed Huddle01 design, this chain-anchored surface includes per-session evidence rather than only node registration, staking, and rewards;

3. a **publicly available research implementation**, in contrast to the closed-and-patented posture;

4. **signed evidence and an explicit consequence workflow**, with a published proposed-penalty taxonomy and forensic summaries for selected events.

§2.4 examines the intellectual lineage of the fourth choice; §2.5 names the resulting gap. The matrix returns in Chapter 6 as the frame for comparing the implemented DVConf mechanisms with its category peers.

2.4 Decentralised Accountability

The production systems surveyed in §2.3 do not provide public per-session evidence linked to an explicit economic consequence. The underlying ideas are not novel: they follow a lineage from volunteer-computing sabotage tolerance through classical and blockchain consensus. DVConf's narrow design extension is an RTC-specific quality-fault class; automatic enforcement of its proposed penalty schedule is not implemented.

2.4.1 Pre-Blockchain: Sabotage Tolerance

Sarmenta's sabotage-tolerance work for volunteer computing [7] predates blockchain slashing. It assigns credibility to volunteer hosts, accepts results through credibility-weighted voting, uses spot checks with known answers, and reduces credibility or excludes repeat offenders when evidence is available. DVConf adopts the evidence-and-consequence pattern, not a direct implementation mapping: a validator-signed `SessionProof` can lead to an on-chain obligation and reputation effect, while coin deduction requires a later cooperative `pay_slash` transaction. ADR-0003 records the resulting project-specific quality-fault design. Sarmenta provides the his-

torical prior art, while Tendermint (§2.4.4) provides the nearer technical prior art.

2.4.2 Classical Consensus Vocabulary

The vocabulary of quorum, Byzantine faults, safety, and liveness follows three foundational protocols. Paxos [30] addresses crash faults and requires $n \geq 2f + 1$. PBFT [9] tolerates f arbitrary faults when $n \geq 3f + 1$, with $O(n^2)$ message complexity. Raft [31] presents crash-fault consensus around a strong leader. ADR-0006 records the project's PBFT-derived per-room design with $f = 1$, four validators, and a three-of-four threshold. The implemented reward path instead requires two independent proofs per relay; the design threshold is not enforced by settlement (§6.2).

2.4.3 Permissionless Trust Roots: PoW to PoS

Bitcoin [32] adds permissionless participation through proof of work; its relevance here is historical rather than a direct penalty mechanism. Ppcoin [33] moves the cost to locked capital and surfaces the nothing-at-stake problem. Later proof-of-stake systems use signed equivocation as evidence for penalties. DVConf borrows the stake-as-economic-commitment pattern for application roles. The relay path implements obligation recording and a later cooperative deduction function; the signalling taxonomy remains design-stage (§3.4).

2.4.4 BFT-PoS and the Slashing Taxonomy

Tendermint [8] combines Byzantine-fault-tolerant consensus with proof of stake and distinguishes evidence of equivocation from failures to participate. Cosmos [34] makes operational penalties governance-tunable. DVConf uses this distinction as design prior art but applies it to RTC

application roles rather than chain validation. Akash [35] supplies a marketplace analogue with staked providers and on-chain settlement. DVConf substitutes latency-and-capacity scoring and scarcity-ratio accounting for Akash's price auction; the validator probe remains composite because some inputs come from relay HTTP (§3.7).

2.4.5 What DVConf Inherits and What It Adds

Table 2.3 summarises the inheritance. Each component has prior art; the surveyed combination and proposed RTC quality-fault class delimit the thesis claim.

Table 2.3: Accountability lineage: inherited components and DVConf-specific deltas.

Component	**Inherited from**	**DVConf-specific delta**
Stake-bearing node registry	Tendermint bonding [8]; Ppcoin staking [33]	Per-role minimum stakes enforced by on-chain registries (§4.2), not chain-validator bonding
Evidence-grounded consequence	Sarmenta credibility deduction [7]; Tendermint equivocation evidence [8]	Signed application-layer `SessionProof`; obligation recording and later cooperative deduction
Severe and liveness taxonomy	Tendermint two-class model [8]	Extended with a third, RTC-specific *quality* class (§2.4.5)
BFT quorum bound $n \geq 3f + 1$	PBFT [9] via Tendermint [8]	Per-room design: four validators and a three-of-four threshold (ADR-0006); implemented settlement: two proofs (§6.2)
Cryptographic non-repudiation	Signature primitives [8], [32]	Dual-signed `SessionProof` under a dual-wallet validator partition (§3.4)
Off-chain work, on-chain settlement	Akash marketplace [35]	Auction replaced by latency-and-capacity scoring and scarcity-ratio settlement (§3.7)

The Third Slashing Class

ADR-0003 extends the two-class taxonomy with *quality faults*: excessive offer-and-answer latency, refusal to serve an authorised client, malformed session descriptions, and an analogous relay-quality surface. These are degraded responses rather than equivocation or complete non-participation. Interactive quality lacks deterministically recomputable ground truth, so the proposed evidence shape uses signed observations rather than a re-executable batch check. The ADR documents a proposed penalty schedule, but its signalling enforcement is not implemented (§6.2). The narrow design claim is therefore an RTC-specific proposed penalty class with a distinct evidence shape, not novelty in slashing, BFT, or staking.

2.5 The Sui Move and WebRTC Gap

The two preceding sections establish two gaps within the surveyed corpus: the production matrix lacks a publicly available research system with signed per-session evidence (§2.3.5), and the reviewed BFT-PoS taxonomies do not define an RTC quality class (§2.4.5). This section names their intersection and bounds the claim.

2.5.1 Why the Substrate Change Matters

Chain interaction is unsuitable for per-packet coordination and remains outside the steady-state per-packet media path. Sui [18] provides the object and transaction substrate used by the prototype, but platform cadence does not establish acceptable join, revocation, or failover latency. Published platform figures remain distinct from the pinned-localnet, polling-inclusive create-and-settle observation reported in §5.2, which does not measure those user-visible paths.

2.5.2 Why the Application Matters

The reviewed chain applications usually concern discrete correctness, whereas a degraded video session is a continuous-valued judgement. The surveyed RTC DePIN systems also stop short of publishing the same evidence mechanism: Huddle01 documents an AnyTrust Orbit L3 and off-chain media architecture [4], [27], [28], while Datagram does not publish its penalty model or evidence shape at mechanism level [5]. Within this surveyed corpus, the combination of a Move contract layer, signed per-session QoS evidence, and an RTC-specific proposed penalty class was not identified.

2.5.3 Bounding the Novelty Claim

The contribution this chapter establishes is narrow and substrate-defined. The following are *not* claimed as novel: slashing (Sarmenta, 2001 [7]); BFT-PoS (PBFT 1999 [9] through Tendermint 2016 [8]); the severe and liveness taxonomy [8]; stake-bearing application validators (Akash [35]); and DePIN media-node networks (Huddle01 [4], Datagram [5]). Within the surveyed corpus, the claim is their combination plus one design extension:

1. an **RTC-specific quality-fault class** whose proposed penalty schedule is documented as a design and whose signalling enforcement is deferred (§6.2);

2. a **dual-wallet validator partition** (§3.4) separating media-plane and staking roles while retaining a public, reused wallet mapping; identity unlinkability is not claimed;

3. **per-session evidence anchoring at the trust root of an RTC architecture** on Sui; its pinned-localnet create-and-settle observa-

tion is reported in §5.2, while mainnet finality and user-visible coordination delays remain open;

4. a **publicly available research implementation** occupying the unrepresented configuration in Table 2.2.

Chapter 3 develops the trust model; Chapter 4 maps it to the implementation; Chapter 5 evaluates selected paths; and Chapter 6 states the remaining limitations.

Chapter 3

Proposed Architecture

3.1 Goals and Non-Goals

The architecture in §3.2–§3.7 is a research prototype rather than a general-purpose decentralised conferencing service. Its goals therefore describe the direction of the design, while the evidence and implementation gaps below bound what the submitted system establishes.

3.1.1 The Four Objectives

The approved proposal defines four objectives.

1. **Resource democratisation.** A four-role registry (relay, signalling, control plane, and validator) admits operators through stake thresholds and role voting rather than an operator allow-list (§3.3). This demonstrates open admission, but not continuous economic bonding, because the implemented workflow does not invoke room-lifetime stake locking.

2. **Operational autonomy.** Assignment and accounting records live on Sui [18], and a control-plane score quorum gates the ordinary room-finalisation path (§3.7). A five-control-plane localnet run reached four-of-five in July 2026, but the evaluated default uses one

control plane, proposals are grouped by score rather than the complete assignment vector, and WAN-distributed operation remains future work.

3. **Enhanced resilience.** The SFU surface contains a cross-relay PipeTransport mesh and a permissionless, heartbeat-staleness-gated `promote_relay` entry. Live runs separately established cross-relay forwarding and later on-chain promotion after verified pre-failure media; they did not establish post-promotion media re-consumption or measure failover MTTR. MCU respawn and automatic client re-attachment remain deferred (§4.6.1, §5.5).

4. **Hyper-local performance.** The scoring library contains a regional term, but the implemented room-event handler supplies an empty `targetRegion`; geographically aware assignment is therefore open. Separately, the wide-area network (WAN) run of 2026-07-05 measured a one-way *lower-bound component sum* of $p_{95} = 70.9$ ms ($n = 30$). This does not validate capture-to-paint latency or regional selection (§5.2).

Decentralised coordination pulls against latency. Assignment, obligation recording, and settlement require a chain round trip, while media forwarding does not. Sui publishes platform-level consensus and checkpoint figures [18]; DVConf instead measures a narrower polling-inclusive SDK-call-to-event path on a pinned single-host localnet (§5.2). The architecture therefore keeps chain-dependent work off the steady-state media path (§4.6.4) without presenting that local observation as a DVConf-specific mainnet-finality number.

3.1.2 The Nine Non-Goals

Table 3.1 records the scope cuts against which the four objectives must be read. Chapter 6 (§6.2) carries the corresponding future-work surfaces.

3.2 System Overview

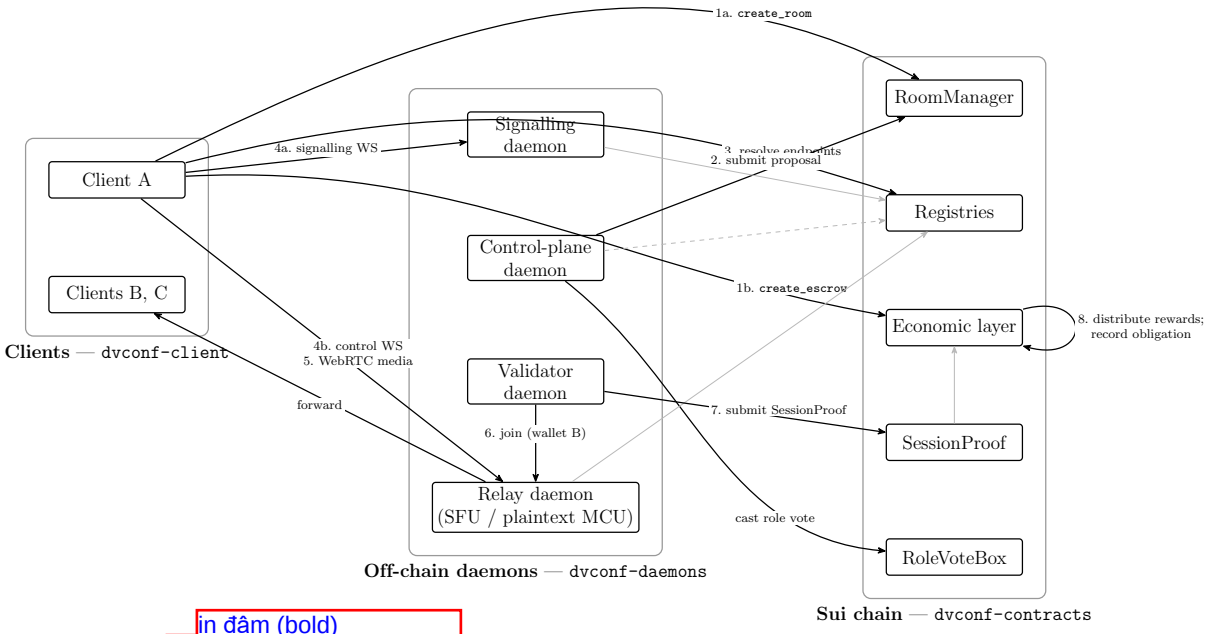
DVConf separates browser clients, off-chain daemons, and Sui contracts (Table 3.2). The chain stores registration, assignment, room-capability, proof, and accounting records but never media bytes. Daemons forward media and react to chain state while retaining explicit local gates, including relay room passwords and TURN-broker authentication. Browsers resolve assignments from chain and control media through relay WebSockets.

3.2.1 The Session Data Flow

Figure 3.1 shows the three layers and the numbered causal paths. A session crosses the three layers in four stages; §3.5 gives the exact transaction and daemon ordering, and the parenthesised numbers refer to the diagram edges.

1. **Establishment and coordination (edges 1–2).** The creator records the room with `create_room` (storing `RoomInfo` in the shared `RoomManager`, accepting no coin) and separately funds a shared `RoomEscrow` with `create_escrow`. Control-plane daemons then submit assignment proposals; a strict score supermajority sets the room `READY`, but the contract installs the first vector in the winning `submitted_score` group rather than a vector proved equal across voters.

2. **Client setup and media (edges 3–5).** The browser resolves assigned relay and signalling endpoints from chain, retaining configuration fallbacks, then establishes WebRTC to the relay over its control WebSocket. SFU forwarding is the primary media mode; the optional ffmpeg MCU path is plaintext-only because compositing requires decoding and cannot operate on an end-to-end-encrypted room (§3.3.1). The standalone signalling daemon is a separate offer-and-answer ser-



in đậm (bold)

Figure 3.1: DVConf separates browser clients, off-chain daemons, and Sui contracts. Numbered edges are the causal paths of the four-stage session flow (§3.2.1): establishment and coordination (`create_room` and `create_escrow` are separate transactions, edges 1a–2), client setup and media (edges 3–5, with the standalone signalling WebSocket distinct from the relay control WebSocket), validator participation (edge 6), and attestation and accounting (edges 7–8). Edge 8's `RelaySlashed` records an obligation event; the stake debit is a separate cooperative `pay_slash` (§3.2.1). Grey edges are supporting registration, heartbeat, and event-subscription paths.

cho font size nhỏ lại, không thì in nghiêng

vice that verifies `RoomCapability`; it is not the live relay's admission boundary.

3. **Validator participation (edge 6).** Validators join through session wallet B and consume media like ordinary peers. The forwarding path performs no validator lookup, so validator status is hidden *at the media plane*, not from public chain correlation.

4. **Attestation and accounting (edges 7–8).** After closure, validators submit dual-signed `SessionProof` objects; the entry verifies

the room, assigned validator, signatures, and that the named relay is registered, but not that it belongs to the room's assignment. A later `distribute_rewards` call requires two stored proofs globally and two distinct validators for each covered relay (§3.4.3); a covered zero-quality relay receives no reward, has its reputation set to zero, and emits `RelaySlashed`. Stake debit is a separate, cooperative `pay_slash` transaction requiring the relay-owned `StakePosition` that no implemented daemon invokes.

The chain event record can be queried without daemon cooperation, but the implemented forensic CLI materialises only proof, obligation, relay-history, and reward reports. It does not reconstruct every authorisation, role vote, pairing step, or off-chain decision.

3.2.2 Properties and Limits Visible at This Level

Five statements bound the overview. **No separate central application database is authoritative for binding room and accounting records**: these records live on Sui, although the evaluated default retains one control plane, AdminCap emergency paths, local gates, and endpoint fallbacks. **Chain means coordination metadata**: video, audio, and ICE payloads remain off chain. **Validator indistinguishability is media-plane scoped**: the wallet-A to wallet-B mapping is public registry state (§3.4.2). **On-chain arithmetic is integral**: role fractions use basis points summing to 10 000. **Registration thresholds are enforced but room-lifetime locking is not**: `StakePosition.locked` exists, yet neither `staking::lock` nor `staking::unlock` is invoked by an implemented application workflow.

3.3 Component Decomposition

The source inventory is organised into eight functional bands and uses six decision labels: KEEP, KEEP + EXPAND, REPLACE, BUILD-NEW, DROP, and OUT-OF-SCOPE. A BUILD-NEW label identifies DVConf-specific code; it does not by itself prove novelty or deployment completeness. Table 3.3 condenses that inventory into thirteen selected component rows and states the implemented boundary of each.

in đậm

Table 3.3: Component decisions and implemented boundaries.

Component	Decision	Implemented boundary
WebRTC plumbing	KEEP `mediasoup` [6]	Integrated ICE, DTLS, RTP, producer, and consumer substrate.
MCU composite	KEEP ffmpeg `xstack`	Optional plaintext-only path; SFU remains the primary media mode, and encrypted rooms refuse compositing.
Relay failover transport	KEEP + EXPAND PipeTransport	Cross-relay forwarding and promotion exist; post-promotion media continuity and the recovery interval are unmeasured.
Simulcast and layer choice	KEEP + EXPAND mediasoup	Implemented three-layer VP8 ladder and `setPreferredLayers`; VP9 or AV1 SVC remains deferred.
TURN and STUN daemon	REPLACE with `coturn` [12]	Selected unmodified verifier; no operational coturn deployment or relayed-media run is claimed.

Continued on the next page

Table 3.3 (continued)

Component	**Decision**	**Implemented boundary**
TURN issuance	BUILD-NEW broker plus event	A single `ControlPlaneCap` mints HMAC credentials and anchors a minimal issuance hash; this is not the room-capability quorum.
Control plane	KEEP and BUILD-NEW	Role voting agrees on an outcome; room pairing agrees only on a submitted score.
Dynamic load balancer	DROP	Partially replaced by one-shot score agreement and heartbeat-gated standby promotion.
Signalling admission	BUILD-NEW, informed by LiveKit [24]	Standalone signalling verifies `RoomCapability`; the live relay WebSocket does not.
Contracts and `SessionProof`	KEEP and BUILD-NEW	Dual signatures and validator binding are enforced; assigned-relay membership is not checked at proof submission.
Forensic CLI	BUILD-NEW	Partial event reports rather than a complete consensus or authorisation replay.
Storage, CDN, recording	DROP and OUT-OF-SCOPE	Room metadata remains on Sui; stored playback is outside the real-time deliverable.
Mobile-native and IoT clients	DROP	No implementation surface.

3.3.1 Media Plane and Opt-In Encryption

The contribution is not a new SFU: mediasoup supplies the forwarding substrate, while DVConf adds assignment, evidence, and accounting around it. Within the implemented simulcast path (Table 3.3), the client selects a consumer layer from viewport and active-speaker state.

Opt-in end-to-end media encryption is layered above mediasoup with the browser Encoded Transform and an SFrame-style frame envelope [25]. When the transform attaches, the relay forwards ciphertext without a media key. This is a scoped property: if encryption negotiation fails, media forwarding continues in plaintext, and the encrypted path is tied to password admission rather than being the default for every room. Because an MCU must decode and composite, the ffmpeg path is structurally incompatible with that encrypted mode.

The PipeTransport mesh similarly has a bounded result. Live runs established cross-relay forwarding before a primary failure and later exercised on-chain promotion while surviving relay processes remained open. They did not assert that clients consumed media after promotion or measure the recovery interval; MCU respawn and automatic browser re-attachment are future work (§5.5).

3.3.2 TURN, Signalling, and Scope Reductions

The TURN software path is implemented: a control-plane daemon can mint an HMAC credential, a relay can retrieve it over authenticated HTTP, and, behind `ENABLE_TURN_DELIVERY`, can return an optional `iceServers` entry. If credential retrieval fails, the relay omits the TURN server entry and continues with the remaining ICE configuration. Secret-rotation primitives also exist, but provisioning and reload against a running coturn instance and end-to-end TURN-relayed media were not demonstrated (§6.2).

Room admission is a different mechanism. The standalone signalling

service caches and verifies `RoomCapability` objects, including room, expiry, revocation, quorum signature, and nonce. The submitted mediasoup relay uses another WebSocket; its join frame carries signature- and nonce-shaped fields without verifying them and can use an optional password established by the first participant. The chain-rooted admission claim therefore applies to standalone signalling, not every browser-to-relay connection.

The DROP classification for the discrete load balancer represents a functional substitution and scope reduction rather than merely a rename. Initial selection and in-place promotion exist; continuous re-orchestration, active geographic targeting, and complete-vector agreement do not. The source reuse matrix is a fuller inventory in which some components receive more than one decision label. Table 3.3 is a thirteen-row, status-updated summary rather than a row-for-row reproduction.

3.4 Trust Model

DVConf anchors registration, room assignment, `RoomCapability`, session-proof, obligation, and reward events on Sui. Public anchoring provides tamper-evident records; it does not make every daemon action chain-authorised, every observation reproducible, or every obligation automatically collectible.

The decentralised cells apply at bounded enforcement points. Standalone signalling verifies the quorum-issued room capability, whereas the live relay does not. Sui exposes proof and economic events, whereas the forensic CLI materialises only a subset. TURN is separate again: one `ControlPlaneCap` can anchor a credential hash, and the shared-secret operational lifecycle remains unproven (§3.6).

3.4.1 Properties of Chain Anchoring

Three narrower properties follow. **Permissionless event access** allows a party with Sui read access to query the relevant package events without a DVConf operator. **Tamper-evident history** prevents a daemon from rewriting checkpointed proof, obligation, and reward records; media behaviour and local decision context still require telemetry. **Public obligation recording** allows reward distribution to set a covered zero-quality relay's reputation to zero, store an obligation in `RoomEscrow`, and emit `RelaySlashed`. Actual debit requires a later `pay_slash` call with the relay-owned stake position; room locking is not enforced, and the implemented workflow does not invoke that transaction automatically.

The cost is a chain round trip for coordination and accounting. Sui's published consensus and checkpoint cadence remain external platform context [18]; §5.2 reports a separate localnet distribution from the SDK submit-call boundary to exact event delivery through a 5 s poller. That result does not convert the platform figures into a measured mainnet control-loop latency.

3.4.2 The Dual-Wallet Validator Partition

Wallet A is the validator's registered identity and supplies one proof signature. Wallet B is a session wallet, fresh per daemon start, publicly associated with A in the `session_wallets` table, and reused until restart or cleanup. It joins the room through the same media path as an ordinary peer and supplies the second signature. Both sign the same BCS-encoded measurement message; `submit_session_proof` verifies both keys and their address bindings.

The partition provides *media-plane indistinguishability*, not anonymity. The relay forwarding code performs no validator lookup, but an observer can correlate the public A-to-B mapping with room joins (§4.3.4, §4.5.4). Proof binding also has a separate gap: submission checks the room and

assigned validator and verifies that the named relay is registered, but does not require that relay to appear in `Room.assigned_relays`. Such a proof contributes to the global stored-proof count even though later per-relay distribution filters for assigned relay IDs.

viết đầy đủ để từ Bound không trở nên "mồ côi" vì chỉ có 1 trên nguyên hàng

3.4.3 Median Reconciliation and the Designed BFT Bound

Project-specific ADR-0006 specifies four validators with a three-of-four threshold, derived from the standard $n \geq 3f + 1$, $f = 1$ Byzantine bound [8], [9]. This internal record documents the implementation rationale rather than serving as independent academic evidence. The implemented settlement rule is narrower: the `QUORUM_THRESHOLD` getter is unused; `distribute_rewards` requires two stored proofs globally and two distinct assigned validators before an assigned relay is covered. Reconciliation is per-relay median, but with exactly two values the result is their arithmetic mean. One deviating validator is therefore neither identified nor rejected, and an out-of-assignment relay ID can inflate the global count. The implementation establishes dual-key verification and a two-proof coverage rule, not enforced three-of-four Byzantine tolerance (§6.2).

3.4.4 What the Model Does Not Claim

The model assumes the underlying Sui network remains available and uncensored; mainnet migration is future work (§6.4). It does not protect against one operator controlling every validator assigned to a room; stake-weighted or geographically diverse assignment remains open (§6.3). A `SessionProof` is a signed QoS observation, not proof of media contents or independently reproducible ground truth. Finally, the live relay join is not chain-authorised: it carries unverified signature and nonce fields and an optional local room password. The implemented surface provides

public coordination and accounting records, dual-key-authenticated validator statements, and two-proof per-relay reconciliation; it does not provide complete replay, BFT settlement, or automatic bond collection (§5.6).

3.5 Session Lifecycle

The seven phases below put the architecture in transaction order: registration; room and escrow creation; score-gated assignment; validator assignment; client setup; media operation and standby promotion; and post-close proof accounting.

3.5.1 Phases 1–2 — Registration and Room Creation

Registration creates a typed capability and role-registry entry. The stake values in Table 3.5 are deploy-time defaults copied into shared state that can be changed through `AdminCap`. Control-plane admission uses a base threshold of 0.5 plus 0.1 for each existing control-plane node; relay, validator, and signalling stakes are validation floors before their role-vote workflows. Validators also register wallet B.

Each operator owns a `StakePosition` initially marked `locked=false`. Package-level `lock` and `unlock` functions exist and unregister refuses a locked position, but neither function is invoked by an implemented application workflow. Registration is therefore stake-threshold-gated without proving room-lifetime stake retention.

Room creation and funding also use separate transactions. The room manager's `create_room` entry stores a pending `RoomInfo` record and emits `RoomCreated`; it accepts no coin and creates no separate shared `Room`. The economic layer's `create_escrow` entry then consumes a coin and creates `RoomEscrow`. The code prevents double distribution but provides no explicit abandonment-forfeit or creator-refund entry, leaving stalled escrow recovery as an operational gap.

3.5.2 Phases 3–4 — Relay and Validator Assignment

Control planes compute candidate scores off chain with a TypeScript replica of the six-term Move formula:

$$\begin{aligned}\text{score} = \frac{1}{\text{BASIS}}\big(&\text{rtt}W_{\text{RTT}} + \text{load}W_{\text{LOAD}} \\ &+ \text{stake}W_{\text{STAKE}} + \text{liveness}W_{\text{LIVENESS}} \\ &+ \text{region}W_{\text{REGION}} + \text{history}W_{\text{HISTORY}}\big).\end{aligned}$$

RTT and load are inverted and normalised, stake is capped, liveness follows heartbeat-age bands, region is a binary match, and history is a 0–10 000 score. Validator-probed RTT removes one self-report path. It does not make proposals wholly verified: the main submit entry does not recompute the score, the room event carries a class hint rather than `expected_participants`, and the handler passes an empty `targetRegion`. Signalling selection separately chooses the lowest load.

The `submit_pairing_proposal` entry groups proposals by equal `submitted_score`. Once a score group reaches the strict threshold—one of one, three of three, or four of five in the exercised fleet sizes—the contract copies relay, validator, and signalling vectors from the first proposal in that group and sets `READY`. Equal scores with different vectors can therefore finalise a vector that other voters did not submit. The five-control-plane localnet establishes that four distinct identities exercised this score-quorum path, not full-vector agreement or PBFT assignment safety. If no score reaches quorum, the creator can call `finalize_room` after the cooldown and at least two proposals; this recomputes scores on chain but introduces creator liveness.

Validator assignment writes registered wallet-B identities into the room. The relay forwarding path treats them as ordinary peers; the public registry association remains visible as described in §3.4.2.

3.5.3 Phases 5–7 — Setup, Running, and Settlement

The browser polls the room assignment and resolves relay and signalling endpoints. The live path joins the relay's mediasoup WebSocket and creates transports there. The standalone signalling daemon is a separate offer-and-answer service whose join can require `RoomCapability`. The relay join presents neither that capability nor `TurnCredentialIssued`; its signature and nonce are unverified, while an optional password established by the first participant is the active admission gate. TURN credentials are an optional NAT-traversal `iceServers` path, not room authorisation, and live coturn traversal is not claimed.

During media operation, each participant uploads one track and receives up to $N - 1$ forwarded tracks; the optional plaintext MCU returns one composite. Validator probes combine direct STUN measurements of RTT, jitter, and loss with relay-HTTP counters for forwarded bytes, peer count, and duration; they do not independently observe the participant media path. Control-plane and signalling daemons schedule 30-second heartbeats, while the relay watcher uses epochs: when the primary heartbeat gap is strictly greater than `MAX_HEARTBEAT_EPOCHS`=3, it can submit permissionless `promote_relay` for another already assigned relay. Standbys are materialised during assignment, not room creation. Existing evidence covers forwarding and later promotion after pre-failure media, not post-promotion client media or MTTR.

The creator's `close_room` call only sets `CLOSED`. Validators then submit dual-signed proofs. A later permissionless `distribute_rewards` call applies the two-proof gates from §3.4.3, partitions proofs by assigned relay, and computes per-relay medians. An uncovered relay receives neither reward nor an adverse obligation. A covered zero-quality relay receives no relay reward, has reputation set to zero, and generates a stored obligation and `RelaySlashed` event. Role-level fractions from `compute_scarcity_ratios` obey tested floors and ceilings, but these are not operator ROI guarantees;

relay payment still depends on coverage, quality, and liveness. Distribution also writes `validator_probed_rtt` for future scoring.

The `RelaySlashed` event does not move stake. `pay_slash` requires a later caller and the relay-owned stake position, and no implemented daemon calls it. Off chain, the control-plane handler can revoke affected room capabilities and make the TURN issuer reject new credentials. It neither rotates the TURN secret automatically nor triggers standby promotion, whose gate is heartbeat staleness.

Finite

3.5.4 The Lifecycle as a State Machine

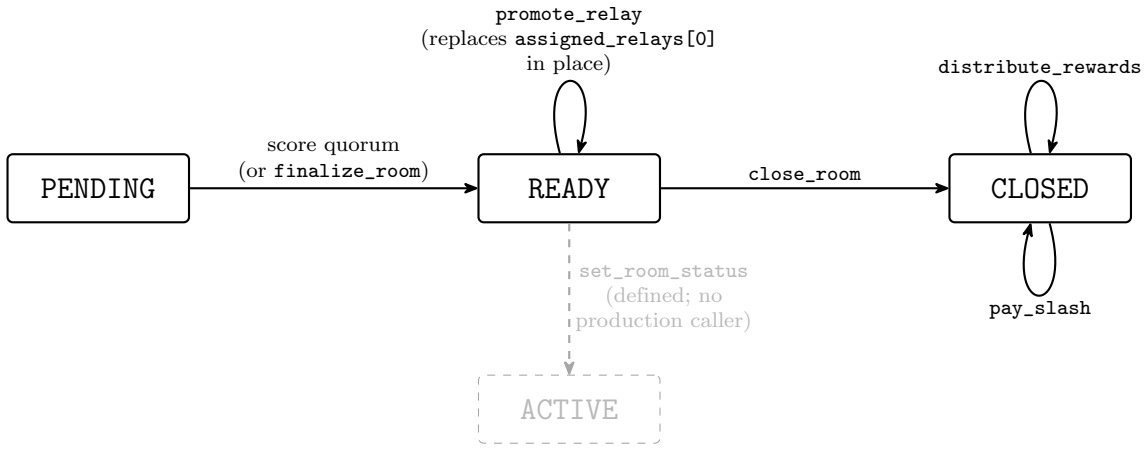


Figure 3.2: The observable room-status lifecycle. The mainline is `PENDING`→`READY`→`CLOSED`; `close_room` can also close any non-closed room. `ACTIVE` is defined by the contract but no production caller invokes `set_room_status(..., ACTIVE)`, so it is never entered (shown dashed). `promote_relay` replaces `assigned_relays[0]` in place without a new pairing quorum or a status change, while `distribute_rewards` and `pay_slash` are later operations that leave the room closed (§3.5.3, §3.4.3).

Figure 3.2 summarises the observable lifecycle. The contract defines `PENDING`, `READY`, `ACTIVE`, and `CLOSED`, but no implemented application workflow invokes `set_room_status(..., ACTIVE)`. The observable lifecycle is therefore `PENDING`→`READY`→`CLOSED`; media running is off-chain state, not an on-chain `RUNNING` value. `promote_relay` replaces

`assigned_relays[0]` in place without closing the room or starting a new pairing quorum. Distribution and any subsequent `pay_slash` transaction leave the room closed.

3.6 Capability Tokens

Two mechanisms that earlier design prose conflated are orthogonal in the implemented system. `RoomCapability` authorises a participant at the standalone signalling service. A coturn-compatible HMAC username and password authorise optional TURN traversal. Their schemas, issuer rules, delivery paths, verifiers, and revocation scopes differ; neither is currently universal client-to-relay admission.

3.6.1 Two Distinct Authorisation Channels

`RoomCapability` binds `room_id`, `peer_pubkey`, role, issue and expiry epochs, issuer quorum, aggregate signature, revocation state, and nonce. Standalone signalling resolves the record, checks room and expiry, verifies quorum signatures, and applies nonce replay protection. A two-of-two issuance was exercised across two localnet hosts in July 2026, but no raw transcript was retained. It establishes cross-process co-signing at that scope, not open-Internet operation, key-bound independence, or economic staking. LiveKit's `VideoGrant` is a conceptual precedent for scoped grants [24], not a field-for-field schema copied here.

The TURN path reuses the coturn REST-API primitive [12]:

$$\texttt{username} = \texttt{expiry:identity},$$
$$\texttt{password} = \texttt{base64(HMAC-SHA1(secret, username))}.$$

`TurnCredentialIssued` records only `cp_miner_id`, `target_miner_id`, issue epoch, TTL, credential hash, and secret ID. It contains no participant, room, publishing, or subscribing grants. TTL is accepted from 900 to 1 800

seconds and defaults to 1 200. Credential bytes travel off chain: the relay can call the control-plane HTTP endpoint during transport creation and return an optional `iceServers` entry. If credential retrieval fails, the relay omits the TURN server entry and continues with the remaining ICE configuration.

3.6.2 Established Properties and Limitations

Revoking a room capability invalidates the standalone signalling cache and closes connections admitted under it. It does not stop an existing mediasoup session because the relay does not enforce that capability. When the TURN issuer marks a relay as subject to an obligation, it rejects new issuance; an existing HMAC credential remains cryptographically valid until its TTL. This response neither rotates the shared secret nor terminates an existing client media session.

Scheduled and emergency rotation mechanisms exist in daemon code, but synchronised provisioning, overlap, and reload against a running coturn instance have not been demonstrated. The on-chain event is also limited forensic evidence: it identifies the issuing control plane, target miner, epoch, TTL, hash, and secret identifier, but cannot reconstruct a participant-to-room authorisation. The forensic CLI has no complete TURN report.

Project-specific ADR-0005 and ADR-0007 document the original combined design and the subsequent broker decision, respectively; they record implementation rationale rather than independent evidence. The implemented claim is correspondingly narrow: quorum-gated standalone signalling plus optional single-control-plane TURN issuance. It does not establish relay-WebSocket binding to `RoomCapability`, quorum approval of TURN, pre-TTL TURN revocation, automatic coturn secret synchronisation, immediate media-session termination, or a complete authorisation replay.

3.7 Consensus Mechanism

The initial proposal specifies a discrete dynamic load balancer, but the implementation contains no such process. In this chapter, *consensus* refers only to two on-chain control-plane tallies: role voting and room-pairing score agreement. Later permissionless accounting is a separate mechanism; pairing does not establish agreement on the complete assignment vector, settlement is not a control-plane ballot, and recording an obligation does not debit stake automatically.

Role votes occur for applications or revotes, not every registry change. Pairing occurs after room creation. `close_room` only closes the room; proofs and `distribute_rewards` follow in separate transactions, and a subsequent `pay_slash` transaction may satisfy a recorded obligation. The same control-plane voters do not reconvene as a settlement quorum.

3.7.1 Evidence from the Control-Plane Threshold

Role voting and pairing use the smallest integer strictly greater than two thirds of the stored `active_cps` membership: one of one, three of three, and four of five in the exercised sizes. That membership is not filtered to control-plane daemons currently emitting heartbeats, so an unavailable registered control plane can affect liveness.

The five-daemon single-host localnet run exercised four distinct voters and reached four of five for role voting and pairing. It establishes execution of that threshold at the tested topology. It does not establish WAN behaviour, economic independence, or Byzantine safety for the full assignment. Proposals are grouped only by score and the first vector in the winning group is accepted; one proposal can therefore influence the selected vector even when four identities agree on the score. PBFT and Tendermint supply the threshold's design lineage [8], [9], not a proof that this score-grouping implementation is PBFT.

Validator settlement has a separate maturity boundary. The four-validator, three-of-four design is not enforced by settlement; the implemented two-proof rule and two-value mean are those in §3.4.3. The implementation provides on-chain supermajority tallying of role outcomes and pairing scores, plus a separate two-proof reconciliation gate.

3.7.2 Scarcity Settlement and Bounded Re-clamping

The `distribute_rewards` entry allocates the escrow across relay, signalling, control-plane, and validator role pools. The implementation pre-clamps each role fraction, redistributes residual basis points among roles not at a bound, and repeats this bounded re-clamping for at most four iterations (§4.6.2). Property tests exercise varied role-count mixes and check that output fractions remain bounded while summing to the distribution total.

These are role-level allocation properties, not an operator minimum return. A relay can receive no reward when proof coverage is insufficient or reconciled quality is zero, and each pool is divided among eligible operators. The result supports a constrained aggregate allocation; it does not establish an ROI floor or complete bribery resistance.

3.7.3 Load-Balancer Substitution and Objective Coverage

The implemented substitute is one-shot score agreement at room establishment followed by permissionless in-place promotion of an already assigned standby after heartbeat staleness. Promotion is not triggered by the later `pay_slash` transaction and does not run a new pairing quorum. Regional weight exists in the scoring library, but the empty `targetRegion` means geographically aware selection is inactive. Chain coordination can remain outside media forwarding. A localnet create-and-settle observer

distribution is measured, but geographically distributed or mainnet coordination and promotion-specific latency remain open.

The event sequence is likewise non-atomic: role and pairing transactions emit outcomes; `close_room` sets `CLOSED`; validators submit proofs; a later distribution call pays eligible rewards and may record an obligation; and a later `pay_slash` could satisfy it. The forensic CLI exposes partial reports rather than a complete consensus transcript.

The objective coverage is therefore partial. Objective 1 is supported by stake-threshold registration and role-voting workflows, subject to unenforced room locking and cooperative deduction. Objective 2 is supported by multi-voter role and score tallies, while the evaluated default remains one control plane and complete-vector agreement is absent; the separate two-host two-of-two evidence belongs to `RoomCapability`, not TURN or pairing. Objective 3 is supported by cross-relay forwarding and a promotion actuator, without post-promotion continuity or measured MTTR. Objective 4 remains open because regional targeting is inactive; the WAN latency result is a separate lower-bound observation.

Two liveness limits remain. A role vote can remain unresolved when no outcome reaches the stored-membership threshold. Pairing can fail to form a winning score group; after a cooldown and at least two proposals the creator has a `finalize_room` fallback, so resolution depends on creator liveness and does not repair score-only agreement. The chapter therefore presents implemented registration, score-gated assignment, proof reconciliation, and standby promotion while retaining complete-vector consensus, enforced three-of-four validator settlement, geographic activation, and measured post-failover continuity as explicit open work.

Table 3.1: The nine explicit non-goals and their manuscript treatment.

#	**Non-goal**	**Treatment**
D1	External professional contract audit	Scoped internal analysis completed; no independent external audit completed.
D2	IoT or edge-scavenging client	No implementation surface; the registry can admit such hardware, but an IoT client is future work.
D3	Mobile-native clients	Browser-only UI; native mobile packaging was not evaluated.
D4	Per-transaction chain logging	Per-session evidence and accounting events; per-packet or per-media-transaction logging is out of scope.
D5	Multi-mixer path distribution	The PipeTransport mesh supplies failover forwarding machinery, not dynamic multi-relay load distribution.
D6	A discrete dynamic load balancer	Replaced by one-shot score agreement plus heartbeat-gated standby promotion; no full-vector consensus or continuous optimisation.
D7	A discrete CDN	Relevant to stored playback, which is outside the real-time-only deliverable.
D8	Continuous per-session orchestration	The implemented control plane assigns once and promotes an existing standby; continuous re-orchestration is deferred.
D9	Capacity stress to hundreds	The evaluated media experiment reached $N = 15$ and supports an $N = 100$ projection. Decode contention is a plausible contributor to the observed increase in upper-tail latency, not a demonstrated explanation of the entire increase. The separate per-IP limit of 10 is an anti-abuse guard, not media-capacity evidence (§5.4).

Table 3.2: The three architecture layers and their responsibilities.

Layer	Repository	Responsibility
Client	`dvconf-client`	Browser UI, wallet integration, and WebRTC peer endpoint.
Off-chain daemons	`dvconf-daemons`	Relay, signalling, control plane, validator, and forensic CLI.
On-chain contracts	`dvconf-contracts`	Registries, room manager, role voting, capabilities, proofs, and economic accounting.

Table 3.4: Trust-root comparison and the bounded DVConf surfaces.

Surface	Centralised trust	DVConf boundary
Recording and forensics	LiveKit Egress, operator-controlled	DVConf `SessionProof` and economic events on Sui
Room admission	LiveKit `VideoGrant`, operator JWT	DVConf `RoomCapability`, issued by a control-plane quorum
Media plane	LiveKit, mediasoup, Janus	No corresponding chain trust root; DVConf media remains off chain
TURN	Application-held coturn secret	Issuance hashes anchored on Sui; coturn operation not demonstrated

Table 3.5: Role-differentiated default stake thresholds.

Actor	**Default stake (SUI)**	**Registry entry**
User	—	`UserRegistry`, `UserCap` only
Relay	0.25	`RelayRegistry`, SFU or MCU mode flag
Control plane	0.5 base, +0.1 per registered control-plane node	`CPRegistry`
Validator	0.1	`ValidatorRegistry`, wallet A and registered wallet B
Signalling	0.05	`SignalingRegistry`, region and load fields

Table 3.6: Reconciliation of the capability design with the two implemented mechanisms.

Concern	`RoomCapability`	**TURN HMAC credential**
Scope	Room, peer public key, and participant role	Optional NAT traversal to a target miner
Issuer	Control-plane quorum and aggregate signature	One holder of `ControlPlaneCap`
Delivery and verifier	Presented to and verified by standalone signalling	Relay calls bearer-protected `POST /turn/issue`; coturn is the intended local verifier
Revocation	Signalling cache invalidation and connection closure	New issuance blocked; existing credential remains valid until TTL
Implemented boundary	Not enforced by the mediasoup relay WebSocket	Broker code exists; coturn provisioning, reload, and relayed media were not demonstrated

Table 3.7: Responsibilities of the three coordination primitives.

Primitive	**Move entry**	**Implemented responsibility**
Role voting	`role_voting`, `RoleVoteBox`	Finalises a role application or revote when its tally reaches the strict-supermajority threshold.
Pairing proposal	`submit_pairing_proposal`	Installs the first recorded vector in an equal-score group when the group's tally reaches the threshold.
Scarcity accounting	`distribute_rewards`	Applies role-fraction bounds and proof coverage after closure; the control-plane set does not vote on it.

vì có nhiều table nên cho thêm mục
3.8. Chapter Summary

Chapter 4

Implementation

4.1 Repository Structure

DVConf comprises three sibling Git repositories: `dvconf-contracts` (on-chain Sui Move), `dvconf-daemons` (off-chain Node.js services), and `dvconf-client` (browser application). Each has its own toolchain and deployment cadence. Boundary formats are inventoried centrally; incompatible changes affecting producers and consumers are released as one coordinated deployment unit (§4.5).

4.1.1 The Three Repositories

The contract build and test commands are `sui move build` and `sui move test`, respectively. The split follows deployment boundaries: Move bytecode is published on its own cadence, daemons restart independently, and the client is deployed as a versioned single-page application. A single release tag across all layers does not fit the Sui upgrade model [18]. State-object Binary Canonical Serialization (BCS) decoders are consequently hand-written in both the control-plane (CP) daemon (`cp-daemon`) and client (§4.5.1).

Table 4.1: The three repositories and their toolchains.

Repository	Language and toolchain	Build artefact	Responsibilities
`dvconf-contracts`	Sui Move; Sui CLI	Published Move package	Registries and room manager; role voting; economic layer; capability tokens
`dvconf-daemons`	TypeScript and Node.js; pnpm, `tsup`, `vitest`	Daemon bundles	Relay and signalling; control plane; validator; forensic tool
`dvconf-client`	TypeScript and React; Vite, TanStack React Query	Static single-page application	Room interface and dashboard; wallet adapter; WebRTC peer

4.1.2 Internal Layout

At contracts commit `17e1fce`, `dvconf-contracts/sources/` contains 26 Move files across seven directories. At their pinned revisions, the daemon workspace contains four long-lived applications, the forensic CLI, and four shared packages; the client has four React Query hooks and four direct or HTTP pollers. `@dvconf/shared` provides RPC access, event polling, types, transaction helpers, logging, and probes, but not state-object BCS decoders. Applications do not import one another. Authenticated claims boards transiently store signed claims, while the pipe mesh carries inter-relay media; neither is authoritative. Room hooks also poll the chain where needed (§4.4.2).

4.1.3 Build Pipeline

Each repository builds independently. All 386 contract tests pass at `17e1fce`; the cost study uses a 20-entry localnet gas run rather than compiler instruction counts (§5.3). The daemon root test is hermetic, with localnet, real-mediasoup, and benchmark suites run separately. Continuous integration (CI) locks only the capability-token boundary; no continuous-

deployment pipeline exists. Clean-room tests and baselines run locally, whereas Chapter 5 identifies Azure-hosted star-topology, inter-relay, capacity, and E2EE comparisons. The single-host validity threat therefore applies only to the relevant results (§5.6).

4.1.4 Cross-Repository Policy

Changes are recorded first in the workspace and then in each affected repository. Contract changes require an Architecture Decision Record (ADR) before the first consuming repository accepts the change. These numbered records are implementation artefacts used for traceability; where cited in this thesis, they identify project decisions rather than constitute primary academic authority. Three source hashes and the workspace hash identify the evaluated state.

4.2 On-Chain Layer

The on-chain layer is one Sui Move package [18], authoritative for registration, room assignment, accepted proofs, reward distribution, and recorded slashing obligations or redistribution transactions. It contains 26 Move files in seven directories.

4.2.1 Modules and Error Codes

The directories `core/`, `access/`, `miner/`, `registry/`, `scoring/`, `security/`, and `audit/` separate responsibilities rather than phases. `core/token.move` defines a DVCONF currency, but staking and settlement use SUI at the evaluated revision.

4.2.2 Abort Codes

Bands are a convention, not an enforced namespace. Of 145 declarations, 142 values are unique; 700, 714, and 882 collide. Code 564 identifies the `room_manager` staleness precondition; in `staking`, 201 is `E_STAKE_LOCKED`, while 202 (`E_NOT_OWNER`) is reserved but unused.

4.2.3 State Objects

`StakePosition` is miner-owned, holds `Balance<SUI>`, and has no implemented locking call path; package-private `staking::slash` returns a coin. `RoomInfo.assigned_relays` begins with the $K = 2$ overlap assignment and may grow through spill authorisation, which appends a registered relay under `ControlPlaneCap`; it also stores participant count and a warm standby. `RoleVoteBox` records miner-role votes at 6667 bps, strictly above two-thirds before integer ceiling. `SlashVoteBox` remains design-only. `SessionProof` stores reduced measurements without signatures (§4.5.4).

4.2.4 Authorisation

Administrative mutations are generally capability-gated. `AdminCap` controls governance, pausing, the standby lifecycle, and assignment fallback; `ControlPlaneCap` gates pairing proposals. Three exceptions are material: `promote_relay` is permissionless but staleness-gated; `distribute_rewards` is permissionless but externally invoked; and proof submission authenticates the sender and registry membership. Pairing quorum counts equal scores, while the first matching proposal supplies the vectors, so different vectors with one score can form a quorum. A 2026-07-01 run exercised four of five control-plane nodes. A separate `QUORUM_THRESHOLD` $= 3$ constant (`constants.move:48`) exists but has no callers and is not connected to the pairing logic (§6.3).

4.2.5 Registration

Generic registration creates `MinerCap` and `StakePosition`; a role-specific entry then inserts the profile. `AdminCap` holders can change the default thresholds. A sixth role would require both a new entry point and changes to centralised unregistration and role-transition cleanup. Role registries snapshot stake at registration and do not synchronise later top-ups or deductions, so selection displays and slashing-obligation calculations can use stale values.

4.2.6 Settlement Economics

`compute_scarcity_ratios` derives four inverse-count shares, iteratively clamps them to $[500, 8\,000]$ bps, and preserves a 10 000-bps sum; adversarial property tests cover the bounds. Distribution is atomic across four registries and uses the two-proof settlement described in §4.5.4.

The zero-quality settlement branch records an obligation, sets reputation to zero, and emits `RelaySlashed`; it does *not* deduct the bond. Deduction occurs only when `pay_slash` receives the accused relay's owned `StakePosition`, with `RelaySlashRedistributed` marking coin movement. The canary-divergence transaction function `slash_for_canary_divergence` has the same custody constraint: it verifies signatures from at least two distinct validators, but the evaluated daemon path has the relay supply its owned `StakePosition` because a validator cannot seize it. With no implemented stake lock or autonomous caller, economic deterrence is cooperative at this revision (§6.3).

4.2.7 Events

Many operational mutations emit typed Move events, forming the principal chain-audit surface. Coverage is incomplete: governance setters and some package mutations emit no dedicated event, and not every Move event has a shared TypeScript mirror or consumer.

4.3 Off-Chain Daemons

`dvconf-daemons` implements four long-lived daemons and a one-shot forensic CLI. Registration, pairing, promotion, and settlement finalise as chain transactions; claims boards over mutual TLS (mTLS) and the inter-relay pipe mesh carry advisory or media bytes but cannot finalise those state transitions. Event coverage is broad but incomplete, and media behaviour cannot be reconstructed from chain events (§4.5.2). Figure 4.1 places these services around the chain.

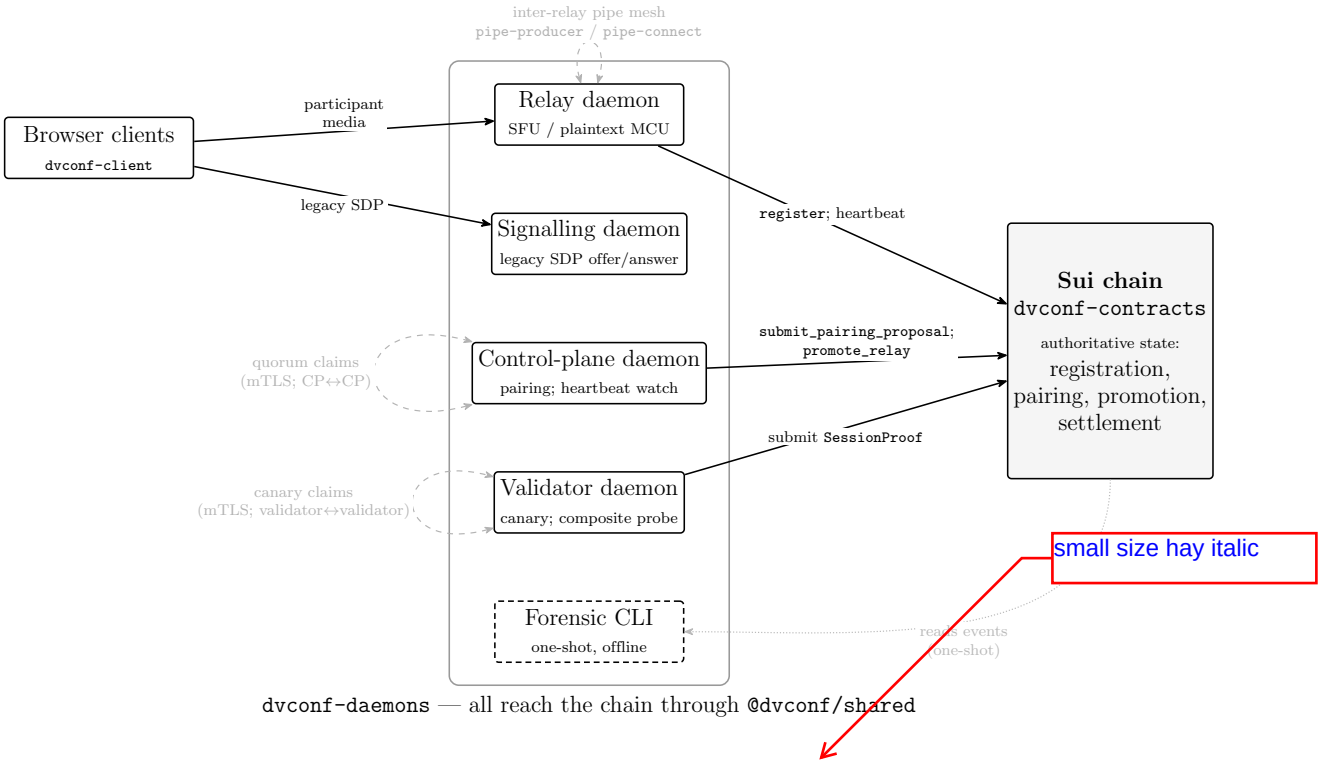


Figure 4.1: The four long-lived daemons and the one-shot forensic CLI around the Sui chain. Solid arrows into the chain are authoritative transactions—only these finalise registration, pairing, promotion, and settlement. The relay is the only daemon that forwards participant media; validators additionally consume canary media (which only relays forward). Dashed edges are non-authoritative carriers: the inter-relay pipe mesh and the two *same-role* mutual-TLS claims boards—the CP-to-CP quorum board and the validator-to-validator canary board (each drawn as a self-loop)—carry media or advisory bytes but cannot finalise a chain decision. The forensic CLI (dashed box) is not a daemon—it performs a one-shot offline read of chain events (§4.3.5).

4.3.1 Relay Daemon

The relay alone *forwards participant media*; validator daemons may consume canary media. A mediasoup Worker [6] provides SFU forwarding over DTLS-SRTP and ICE [11], [15], [16]; `ffmpeg xstack` provides plaintext MCU composition. July 2026 cross-host evaluations exercised PipeTransports with a WebSocket handshake; `pipeToRouter()` remains same-process only. SFrame [25] makes the relay content-blind only on legs to which the client-side encoded-transform shim attaches successfully; this shim fails open (§4.4.1), and the MCU rejects E2EE because composition decodes frames. The LiveKit comparison [24] is bounded to the May 2026 survey. A primary is promotable when its heartbeat gap *exceeds* three epochs. Pairing uses the CP proposal path; standby lifecycle remains `AdminCap`-gated (§6.3).

4.3.2 Signalling Daemon

Browser media signalling uses the relay WebSocket; the standalone daemon retains legacy SDP offer-and-answer signalling and discovery. Room passwords gate admission, claims boards use bearer authentication, and capability tokens gate chain entries. The limit of 10 connections per IP address provides rate limiting rather than defining a capacity ceiling (§5.4). Automatic client reattachment after promotion is not implemented.

4.3.3 Control-Plane Daemon

State from `RoomCreated` and `EscrowCreated` feeds a TypeScript mirror of the six-term on-chain score; each CP ranks candidates and submits `submit_pairing_proposal`. `RoomManager` groups equal-score proposals and finalises at the active-CP threshold; `RoleVoteBox` is a separate miner-role voting loop. The five-CP run exercised score equality with four of five nodes. `RelayHeartbeatWatcher` separately calls permissionless `promote_relay` after the chain's staleness predicate passes, without

reattaching client media. The older four-phase detector has offline tests but no runtime importer. The cp-daemon subset has 421 tests; two failures outside it cause the aggregate command to fail.

4.3.4 Validator Daemon

The daemon uses two wallets, giving media-plane indistinguishability only, and measures through a composite STUN and relay-HTTP probe whose relay-reported counters the signatures do not independently verify (§4.5.4). The canary-divergence function requires two distinct validators, yet bond custody means the evaluated path demonstrates cooperative, relay-initiated payment rather than validator-enforced deduction (§4.2.6).

4.3.5 Forensic CLI

`collect` performs a one-shot drain of its selected event queries: it starts from a null cursor and truncates its JSON Lines (JSONL) output, so it has no persisted resume state. `report` summarises that transcript without further chain access. Its proof report uses room-wide counts and a room-wide median rather than the implemented per-relay settlement procedure, and it cannot reverify omitted signatures; its reward cross-check is not implemented. The tool requires neither a live chain nor media bytes at report time, but it does not provide a complete settlement or failover replay.

4.3.6 Shared Observability

`@dvconf/shared` provides chain access, types, constants, probes, carrier security, and structured logging, and `BENCH_LATENCY=1` enables probe writes. The benchmark and forensic trace-identifier semantics are stated in §4.5.5.

4.4 Client

The React 18 and Vite client exposes `HomePage`, `RoomPage`, read-only `DashboardPage`, and `OperatorPage`. It combines React Query with direct polling rather than routing all chain access through one mechanism.

4.4.1 Pages

`RoomPage`, built on mediasoup-client [6], owns relay resolution, WebSocket and transport lifecycles, and media production and consumption. In E2EE mode it attaches SFrame transforms [25] to each leg; content blindness holds only where attachment succeeds. The shim fails open, and the badge reports the requested mode rather than successful per-leg attachment. The password is admission material; the media secret travels in the invite-link fragment. `DashboardPage` has fourteen isolated panels, while `OperatorPage` is a direct-URL operator interface.

4.4.2 Hooks

Four dashboard hooks for active rooms, the registry, economics, and voting use React Query; the registration timeline and chain events use separate intervals; canary coverage and daemon health poll HTTP endpoints. Top-level room hooks own media and wallet lifecycles and also perform direct event polling and `dev-inspect` queries. Error boundaries contain decoding failures but do not prevent schema drift. At `d7a622c`, 545 tests pass and one is skipped.

4.4.3 Wallets

On public networks, Sui dApp Kit [18] uses extension wallets; localnet demos use funded development keypairs. Validator session wallets are server-side and never instantiated by the client.

4.4.4 Boundary Concerns

The client owns a BCS mirror independent of the cp-daemon (§4.5.1), and event queries are scoped by package and module. DVConf implements TURN credential issuance, secret rotation, control-plane generation and RPC retrieval, relay fetching, and optional `iceServers` delivery. The evaluation did not demonstrate live coturn [12] provisioning or reload, coturn-side eviction, or TURN-relayed media; delivery failure falls back to direct connectivity.

4.5 Cross-Cutting Concerns

This section describes state mirrors, events, WebSocket messages, the dual-signed proof, and the two Chapter 5 record families. Changes are coordinated across the components that consume them; only incompatible changes require a breaking migration.

4.5.1 BCS State Mirrors

Sui BCS [18] is the byte-layout contract between a serialised Move object and each decoder. Two independent hand-written sets exist: cp-daemon state schemas and client dashboard schemas. `@dvconf/shared` contains TypeScript types, not these decoders. A struct change must reach both sets manually; CI locks only the capability-token boundary.

4.5.2 Events and Cursors

Move events are the principal chain-audit surface, not complete write coverage; daemon metrics and health endpoints are operational surfaces rather than chain-auditable. Module-scoped daemon pollers persist the last page cursor, but handlers run before the cursor is saved: a crash may replay a page prefix, giving at-least-once delivery. When pending events exceed a configured backlog limit, the replay logic may advance the cursor past older pages to return within that limit; those older events are then omitted

rather than delivered. Consumers therefore require idempotent handlers and cannot assume a gap-free history. Client cursors remain in memory. Forensic `collect` instead starts from null and truncates its output on every invocation. Trigger provenance is incomplete and no forensic failover report exists.

4.5.3 Signalling Protocols

The WebSocket inventory is directional (Table 4.2). Room passwords gate admission, claims boards use bearer credentials over mutual TLS [13], and capability tokens gate chain entries; an admitted session has no additional per-message authentication. Promotion drops the existing socket, but automatic rejoining and media resubscription are not implemented.

Table 4.2: Directional WebSocket message inventories.

Protocol and direction	**Types**	**Contents**
Client → relay (mediasoup session)	10	`join`, `leave`; transport lifecycle; `produce`, `consume`; simulcast and consumer control; opaque E2EE key bundle
Relay → client	9	Router capabilities and room mode; transport, producer, and consumer acknowledgements; `newProducer`, roster, active speaker, and error
Relay ↔ relay (pipe mesh)	2	`pipe-producer`; `pipe-connect`
Client ↔ legacy signalling (SDP offer-and-answer)	5 inbound; 4 outbound	Inbound: `join`, `offer`, `answer`, `ice-candidate`, and `leave`; outbound: `welcome`, relayed offers, answers, and candidates

4.5.4 The Dual-Signed Session Proof

`SessionProof` is a settlement input with three layers (Figure 4.2). Both wallets sign the identical nine-field BCS measurement message; public keys and Ed25519 signatures travel as transaction arguments and are verified through the BLAKE2b-256 public-key-to-address binding; the stored struct retains no signature material. The measurement is composite: direct STUN supplies RTT, jitter, and loss, while the relay HTTP endpoint supplies bytes, peers, and duration. Signatures authenticate the validator's attestation but do not independently verify relay-reported counters. Wallet B is generated at each daemon start, publicly mapped to A, and reused until restart or cleanup, so the property is media-plane indistinguishability only (§6.3).

Settlement requires two distinct validator proofs; with two, the value labelled "median" is their arithmetic mean, and the designed three-of-four threshold is not connected to the implemented settlement logic. Submission authenticates an assigned validator and a registered relay but does not require the named relay to belong to `RoomInfo.assigned_relays`. Such a proof is stored and enters validator scoring with a zero fallback median. Byzantine tolerance and settlement integrity against an assigned malicious validator are therefore not established at this revision (§6.3).

4.5.5 Record Families

Benchmark `LatencyEvent` and forensic `ForensicLine` are distinct JSONL families with no cross-family key. Unseeded benchmark writers use process-local trace identifiers; cross-process correlation requires a common caller-supplied `BENCH_TRACE_ID`. Forensic rows carry no trace identifier. Raw forensic payloads reproduce the implemented summaries, not settlement invariants or omitted signatures.

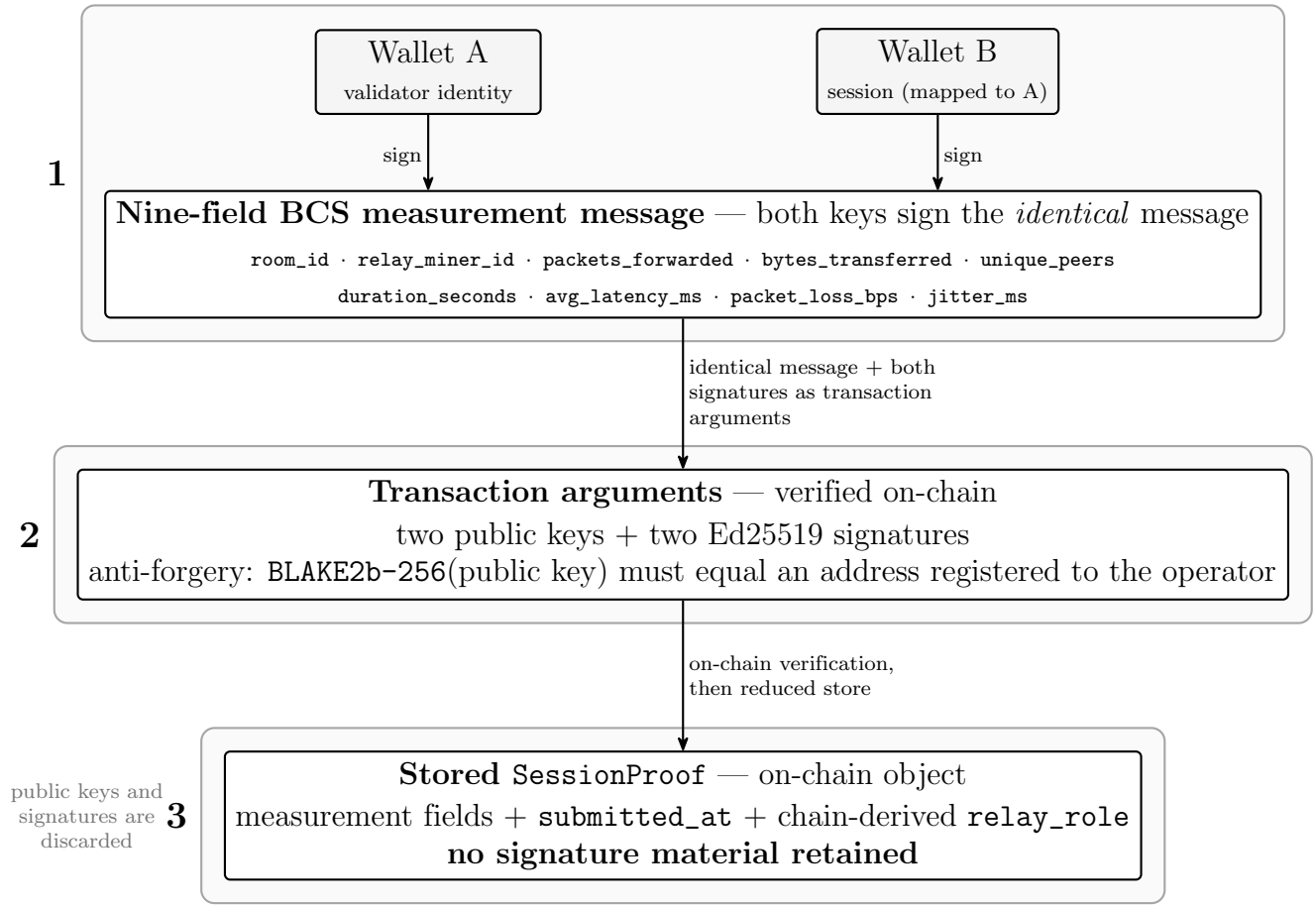


Figure 4.2: The `SessionProof` settlement input has three layers. **(1)** The validator daemon's two wallets—Wallet A (its registered identity) and Wallet B (a per-start session wallet publicly mapped to A)—both sign the *identical* nine-field BCS measurement message; public-key bytes are not part of the message. The measurement is composite: STUN supplies round-trip time, jitter, and loss, while the relay's HTTP endpoint supplies bytes, peers, and duration. **(2)** The two public keys and two Ed25519 signatures travel as transaction arguments; anti-forgery derives from the `BLAKE2b-256` public-key-to-address binding. **(3)** The chain stores a reduced struct with no signature material. This dual signing binds one validator's two keys and is distinct from settlement, which requires two *distinct* validators' proofs; signatures authenticate the attestation but do not independently verify the relay-reported counters (§4.5.4).

small size or italic

4.5.6 Change Management

An ADR names affected consumers; incompatible changes affecting producers and consumers are released together; non-backward-compatible layouts carry a migration note; and historical event readers must tolerate removed fields. Apart from one capability-token CI lock, these controls rely on a manual checklist.

4.6 Engineering Trade-offs

Four significant implementation choices are examined here; unresolved work is consolidated in Chapter 6.

4.6.1 Hybrid SFU and MCU on the Same Relay

Each relay implements a mediasoup SFU [6] and a plaintext `ffmpeg xstack` MCU in one artefact. The SFU is the default path; the MCU rejects SFrame E2EE [25] because composition decodes frames. This supports two room configurations at the cost of a second process path and an MCU failover path that is not implemented. The 8–15 s SFU interval is a design budget, not a measured mean time to recovery (§5.5.5).

4.6.2 Iterative Scarcity Re-Clamping

Four inverse-node-count shares must sum to 10 000 bps while each remains in $[500, 8\,000]$. The implementation clamps, redistributes only over unbound roles, and re-clamps for at most four passes; adversarial property tests verify both bounds and the exact sum. The bounded iteration satisfies the constrained normalisation at the cost of additional implementation complexity.

4.6.3 Operations under AdminCap Control

The evaluated pairing path uses `submit_pairing_proposal` under `ControlPlaneCap`; stale-primary promotion is permissionless. The five-member control-plane run evaluated the score-equality path with four of five nodes. The deployment key still controls standby setup and manual swap and replacement operations, so the migration is partial and the residual centralisation is material (§6.3).

4.6.4 Chain Authority with Explicit Carriers

Registration, pairing, promotion, and settlement finalise as Sui transactions. Claims boards and the pipe mesh carry advisory or media bytes without finalising those transitions. Chain objects and effects are readable without daemon logs, but events omit some writes, forensic reports do not reverify settlement invariants or signatures, and the chain cannot reconstruct media behaviour. A pinned-localnet campaign measures SDK-call-to-event delivery for room creation and reward distribution through the unchanged production poller (§5.2); it does not measure `RelayPromoted` visibility or complete recovery, so no quantified failover headroom is claimed.

4.6.5 Remaining Limitations

The cross-host PipeTransport mesh has been implemented and evaluated between hosts; the interval from relay swap to pipe readiness remains unmeasured. Chapter 6 consolidates the remaining work: MCU respawn, automatic client reattachment, the residual MCU integration case, three failover operations unmeasured in the cost study, standby-lifecycle capability migration, forensic failover reporting, BCS drift, and live coturn integration [12].

Two security limitations require architectural changes: proof submission does not bind the named relay to the room assignment, and bond cus-

tody makes quality-based and canary-based deductions cooperative rather than autonomously enforceable. Mean-of-two settlement and stale registry stake snapshots further bound accountability. The `SlashVoteBox` proposal recorded in ADR-0003 remains design-only; none of these mechanisms is treated as implemented enforcement (§6.3).

Chapter 5

Evaluation

5.1 Methodology

This chapter evaluates DVConf across four evaluation dimensions: end-to-end **latency** (§5.2, against a sub-200 ms design target), on-chain **cost** per function and for one executed $K = 2$, $N = 4$ pinned-localnet session (§5.3), kept distinct from the proposal's preliminary room-creation estimate, **capacity** relative to the proposal-reported 5–10-participant multi-room scale and its Phase 2 beyond-range goal (§5.4), and relay **failover** as an operationalisation of the proposal's self-healing objective (§5.5). Each dimension is evaluated as an independent workload with its own scenarios, instrumentation, and interpretation criteria. The resulting artefacts use dimension-specific JSONL record types (§5.1.4) and family-specific parsers; no universal cross-family envelope is implemented.

5.1.1 Test Environment

The single developer workstation (Windows 11, Intel Core i9-12900HX, 32 GB RAM) hosts the baseline stack: the contracts on a local Sui validator, the daemon workspace under pnpm, and two Chromium browser clients. The localhost latency proxy (§5.2.5), the E2EE loopback run (§5.2.8), and the pinned-localnet chain-observer campaign (§5.2.4) are

single-host experiments on this machine; the chain campaign additionally uses one persistent localnet with sequential shared state and warm caches. Four experiments use external infrastructure. The wide-area component-sum experiment (§5.2.6) ran both Chromium peers on the laptop against one Azure relay; the inter-relay experiment (§5.2.7) spanned two Azure relay hosts; the July capacity and E2EE-versus-plaintext experiments (§5.4) used a Korea–Japan two-relay topology with a separate, co-located synthetic-consumer host; and the matched cloud-WAN E2EE comparison (§5.2.8) placed the producer client in Japan East, the consumer client in Korea Central, and the relay with its signalling and localnet services on a separate Korea Central host. The virtual machines of the first three campaigns have since been decommissioned; the matched cloud-WAN window's three virtual machines remained provisioned at the time of writing, with their inventory recorded in the run manifest.

The single-host baseline biases the loopback latency figures downward because it omits a LAN switch and data-centre fabric. The cost figures in §5.3 are standalone per-transaction measurements that include each transaction's own overhead; cross-transaction batching and amortisation are not evaluated. The wide-area runs remove the loopback-network bias, but retain other downward biases. The component-sum experiment (§5.2.6) uses a synthetic camera, places both peers on one laptop and access network, and derives one-way delay from half the round-trip time. The inter-relay run (§5.2.7) similarly estimates one-way delay on a single link in one run. Section 5.6 records these limitations. The component observations are valid within their stated harnesses, but the full capture-to-paint latency criterion remains unresolved.

Table 5.1: The four evaluation dimensions and their scenario inventories.

Dimension	**Source methodology**	**Scenarios**	**Primary output**
Latency (§5.2)	Latency-decomposition methodology	Same-region baseline, MCU design comparison, cross-region component, loaded room, and pinned-localnet chain observation	Component-sum estimator with p_{50} and p_{95}; per-component decomposition
Cost (§5.3)	On-chain gas measurement (localnet, pinned protocol)	20 measured function rows; executed $K = 2$, $N = 4$ trajectory; historical $K = 1$ projection	Per-function gas and measured pinned-session cost
Capacity (§5.4)	Relay-saturation stress campaign	Single-room viewer sweep and multi-room concurrency	Acceptance rate, fan-out latency, and failure code
Failover (§5.5)	Failover recovery-time methodology	In-process warm-pipe cutover, relay termination, chain promotion, and detector scenarios	Mechanism-floor timing plus observed promotion; full kill-to-render recovery unmeasured

5.1.2 Evaluation Dimensions

5.1.3 Instrumentation

Three layers carry instrumentation. **Off-chain daemons** expose a `LatencyProbe` hook gated by `BENCH_LATENCY=1`; when disabled, it creates neither a sampler nor a JSONL writer. The relay reads RTCP receiver-report `roundTripTime` from RTP-stream statistics rather than `Transport.getStats()`. For the inter-relay input in §5.2.7, the value is read from the standby's piped `Producer inbound-rtp` statistic and halved. The separate relay-to-client `L_relay_fwd` probe reads the `Consumer outbound-rtp` round-trip time. A dedicated signalling ping-and-pong probe times the control path independently of the production WebSocket heartbeat, and

the control-plane daemon instruments its scoring loop. The daemon also hosts the legacy failover-detector state machine evaluated in §5.5; this analyser is distinct from the implemented `RelayHeartbeatWatcher` promotion trigger (§5.5.3). The validator daemon times a complete per-room measurement cycle, including probes of all assigned relays, proof construction and signing, and any applicable escrow transactions. Consequently, `L_validator_check` depends on topology and chain state rather than representing a single round-trip. **On-chain**, Sui Move events provide the principal failover-observation surface, while the cost study reads transaction `effects.gasUsed` from a pinned localnet (§5.3). The chain-latency campaign wraps the unchanged shared production `EventPoller` with an evaluation handler that records external monotonic timestamps and exact digest, full-event-type, and room matching; the poller itself is not modified (§5.2.4). **Client-side**, WebRTC `getStats()` supplies the observations used by the assembled estimator. The synchronised-clock visual-marker method required for direct capture-to-paint measurement was specified but not performed.

Two instrumentation properties are relevant. First, when `BENCH_LATENCY` is unset, no measurement sampler or JSONL writer is created. When enabled, RTP-stream polling and synchronous JSONL output consume CPU, but no calibration between enabled and disabled probes or retained timing artefact quantifies that observer effect; no numerical overhead bound is therefore claimed. Second, a caller-supplied `BENCH_TRACE_ID` can correlate records across processes. When it is unset, each `LatencyWriter` resolves a fresh UUID, so cross-process correlation requires the same supplied identifier at every participating writer. Forensic records instead use transaction and event-sequence identifiers, and the current tooling provides no cross-family trace join.

5.1.4 Observation Schema

The measurement artefacts use separate JSONL record families rather than a shared envelope. Latency-benchmark records use `schema_version=1.0`, a numeric `ts`, and the fields `trace_id`, `scenario`, `source`, `instance`, `metric`, and `value_ms`. Forensic records use `schema=forensic-cli/1.0`, an ISO-8601 `ts`, and the fields `tx`, `seq`, `module`, `event`, and `payload`; they contain neither `trace_id` nor `scenario`. The chain-latency family stores one run-metadata row followed by metric rows, plus a separately hashed exact-event transcript and a manifest that binds both files, the source status, and the run log. The filtered wide-area analysis set contains 2,760 records for 30 intended sessions after the documented exclusion of 116 earlier smoke-test and diagnostic rows. The unfiltered 2,876-record capture, hashes, selection rule, and replay command are retained. Replaying the `join-g2g` assembler reproduces the reported p_{50}, p_{95}, and p_{99} values of 64.8, 70.9, and 72.0 ms. This remains a reduced-fidelity lower bound because real-camera capture is absent, L_{present} is treated as zero, and the 12.5 ms display scan-out residual is a fixed modelling constant.

5.1.5 ~~Out of~~ Scope

of DVConf

Four categories are not evaluated. Application-level join time conflates on-chain coordination with WebRTC negotiation and was not assigned a proposal target. Absolute mainnet gas is not reported: Section 5.3 measures localnet functions and one pinned $K = 2$, $N = 4$ session, then converts the result under a labelled USD-per-SUI scenario; three unimplemented signalling-penalty terms remain estimates. Consensus and mainnet finality are not reported either: the local chain distributions in §5.2.4 include a five-second polling cadence and are not browser, join, revocation, promotion, or recovery measurements. Subjective quality of experience, such as mean opinion score and perceived smoothness, is also outside the scope of this study.

5.2 Latency

5.2.1 Target

The proposal specifies **sub-200 ms end-to-end latency** as a design target. ITU-T Recommendation G.114 places one-way mouth-to-ear delay up to 150 ms in the "good" range and identifies 400 ms as a general planning limit, while noting that highly interactive applications may be affected below that limit [17]. This thesis adopts 200 ms as an engineering ceiling for a two-peer, same-region SFU baseline over broadband, without recording or transcoding. It is a project-specific target contextualised by G.114 rather than a perceptual boundary defined by the recommendation. The executed experiment provides only lower-bound evidence against this target; it does not constitute direct capture-to-paint validation.

5.2.2 Definitions

Latency is considered at two levels. *Component-level* costs isolate a signalling, relay-path, daemon, or chain interaction; six are defined in Table 5.2. The two chain rows are pinned-localnet, polling-inclusive application observations rather than media or mainnet-finality measurements. *End-to-end* glass-to-glass latency is the elapsed time from camera-frame capture to display presentation and is the only measure that maps directly to the design target:

$$L_{\mathrm{g2g}} = L_{\mathrm{cap}} + L_{\mathrm{enc}} + L_{\mathrm{send}} + L_{\mathrm{fwd}} + L_{\mathrm{recv}} + L_{\mathrm{dec}} + L_{\mathrm{present}}.$$

L_{g2g} is not directly available from a daemon, so three estimators are distinguished. The planned full **Option A** is a synchronised-clock visual-marker measurement of the capture-to-paint path; it was not performed. The executed **Option B** derives an estimate from WebRTC `getStats`: half the ICE candidate-pair round-trip, jitter-buffer delay, and a constant 50 ms

Table 5.2: Component-level latency symbols.

Symbol	Definition	Observation point	Evidence/status
`L_sig_rtt`	Signalling↔client WebSocket round-trip	Signalling daemon	Local loopback, one run: $n = 48$; $p_{50} = 1$, $p_{95} = 3$, $p_{99} = 3$ ms
`L_relay_fwd`	Relay-to-client RTCP receiver-report round-trip time from the Consumer's `outbound-rtp` statistic; a network-path observation, not in-router processing	Relay daemon	WAN RTT: $n = 118$; $p_{50} = 96.71$, $p_{95} = 117.49$, $p_{99} =$ 296.71 ms
`L_chain_create`	SDK submit-call start→exact `RoomCreated` delivery	Shared production event poller	Localnet, $n = 30$; $p_{50} = 5003$, $p_{95} = 5018$, $p_{99} = 5020$ ms
`L_chain_settle`	SDK submit-call start→exact `RewardsDistributed` delivery	Shared production event poller	Localnet, $n = 30$; $p_{50} = 4743$, $p_{95} = 5120$, $p_{99} = 5184$ ms
`L_cp_score`	Control-plane scoring loop: read registries→ranked candidate set	Control-plane daemon	No valid full-loop distribution
`L_validator_check`	Full per-room validator measurement cycle: probe all assigned relays, build and sign proofs, and, when an escrow exists, submit and await the corresponding transactions	Validator daemon	Topology- and chain-state-dependent; not measured here

placeholder for capture, encoding, decoding, and rendering. The localhost run uses a narrowed variant that omits the jitter-buffer term because of a `wrtc` unit defect. This omission biases the proxy downward, while the total estimation error remains uncalibrated. The wide-area results below instead use a component-sum estimator constructed from browser `RTCStats`.

5.2.3 Scenarios

Table 5.3: Latency scenarios, ordered by complexity.

Scenario	**Topology**	**Network**	**Primary metric**
Same-region baseline	Two-peer SFU	Same region, broadband	$p_{50}(L_{\text{g2g}})$ and $p_{95}(L_{\text{g2g}}) <$ 200 ms
MCU comparison	Four-peer MCU composite	Same region	Design only; see below
Cross-region component	Two-peer SFU	Cross-region path	Cross-region increase
Loaded room	One-room SFU with increasing N	Same region	$L_{\text{fwd}}(N)$ and $L_{\text{g2g}}(N)$

The same-region baseline is the target-bearing reference scenario. The end-to-end cross-region call was *not conducted*; its framing is informed only by the separately measured relay-to-relay component in §5.2.7. A controlled impairment sweep also remains future work (§5.6), while the loaded-room scenario is evaluated in the capacity study (§5.4). **The MCU comparison was not measured.** An MCU must decode media to composite it, which is incompatible with the implemented content-blind encrypted path (§2.2.3); simulcast provides downstream adaptation without compositing. The SFU and MCU comparison is therefore addressed at design level, and no MCU latency figure is claimed.

5.2.4 On-Chain Coordination Latency

DVConf anchors session state in Sui shared objects. A dedicated pinned-localnet campaign measures two application-observation paths. For each of 30 sequential successful rooms, monotonic timing starts immediately before `SuiClient.signAndExecuteTransaction` and stops when the unchanged shared production `EventPoller` delivers the exact event to an evaluation handler. Matching requires the transaction digest, full event type, and room identity. The poll interval is 5,000 ms. For `RoomCreated`, `L_chain_create` has $p_{50}/p_{95}/p_{99} = 5003.2/5017.9/5020.4$ ms. For `RewardsDistributed`, `L_chain_settle` has 4742.6/5120.2/5183.7 ms. The write-once bundle and deterministic replay retain SHA-256 provenance for the raw records.

The samples share one persistent single-host localnet, a $K = 2$, $N = 4$ roster, one registered user, registries, process, RPC endpoint, execution order, and warm caches; they are not independent sessions. The distributions include the configured polling cadence. Auxiliary transaction-return timestamps record local `waitForTransaction`/`getTransactionBlock` availability, not consensus finality. The result is therefore not a browser or React UI measurement, mainnet or consensus finality, join or revocation delay, `RelayPromoted` visibility, or failover recovery time. Room creation and reward settlement remain outside the per-frame media path, and the unmeasured user-visible quantities are not added to the interactive-latency result.

5.2.5 Recorded Results

Table 5.4 distinguishes direct observations from model inputs. The signalling row is a same-process loopback floor; the media baseline is a reduced-fidelity component sum; and the two cross-region rows are separate path observations, not an end-to-end call. The MCU comparison is resolved at design level (§5.2.3), and the loaded-room scenario is evaluated in the

capacity study (§5.4).

Table 5.4: Latency observations and model inputs. Signalling is a same-process loopback floor; localhost media adds 50 ms to half the round-trip time; and the WAN baseline is a reduced-fidelity one-way sum. Relay-to-client is path RTT and inter-relay is separately halved; neither is end-to-end latency.

Scenario	p_{50} **(ms)**	p_{95} **(ms)**
Local signalling `L_sig_rtt` (same-process loopback)	1	3
Same-region localhost proxy (constant floor)	50	50
Wide-area baseline (one-way component-sum estimate)	**64.8**	**70.9**
Relay-to-client `L_relay_fwd` (RTCP-RR RTT)	96.71	117.49
Cross-region inter-relay component $t_{\text{hop,net}}$ (model input)	34.5	35.0

A retained local run connected eight synthetic WebSocket peers to the production signalling server in one Node.js process. Six samples per socket yielded $n = 48$ `L_sig_rtt` observations: nearest-rank $p_{50}/p_{95}/p_{99} = 1/3/3$ ms, with mean 1.46 ms. These temporal samples are nested within eight sockets in one run, not 48 independent sessions; same-process, same-event-loop loopback makes the distribution a control-path mechanism floor rather than WAN or deployment latency.

Within the retained Malaysia West–Korea Central two-relay run, the consumer leg from relay B in Korea Central to the laptop in Vietnam produced $n = 118$ temporal `L_relay_fwd` RTT observations. Its nearest-rank $p_{50}/p_{95}/p_{99}$ values were 96.71/117.49/296.71 ms, with mean 103.43 ms. This is path-specific round-trip time, not a one-way delay, 118 independent geographic paths, or relay in-router processing. Its p_{95} does not meet the earlier < 50 ms WAN design expectation; the direct distribution therefore

replaces that expectation as the reported result.

A localhost loopback run used the narrowed Option-B proxy: 50 ms plus half the round-trip time. Across five 60-second windows ($n = 600$), both p_{50} and p_{95} were 50 ms. This is the constant floor of the proxy, not a network measurement: the loopback ICE round-trip rounds to zero, and the proxy omits the jitter-buffer term. The run establishes operation of the measurement pipeline and near-zero loopback round-trip time, but does not measure most of the media path or real-Internet capture-to-paint latency. The first two rows of Table 5.4 therefore use different methods and are not directly comparable.

5.2.6 Wide-Area Component-Sum Latency Estimate

The wide-area campaign (run `wan-20260705T060736`) used one Azure relay and measured additional browser-pipeline components, while capture and presentation remained unresolved. Both Chromium peers ran on one laptop and access network against the relay, so both candidate-pair round-trip times traversed the wide-area path. Per-component `RTCStats` for encoding, jitter buffering, and decoding were assembled offline into a one-way sum. Each network leg was estimated by halving its measured round-trip time, and the synthetic camera excluded capture. Across $n = 30$ sessions, aggregated as per-session medians, the one-way estimates were $p_{50} = 64.8$ ms, $p_{95} = 70.9$ ms, and $p_{99} = 72.0$ ms. At this sample size, p_{99} is close to the maximum and is not treated as a population-tail estimate; the reported result is therefore p_{50} and p_{95} for these 30 sessions.

The per-session median component breakdown attributes approximately 39 ms to the two estimated one-way network legs and approximately 26 ms to encoding (3.1 ms), jitter buffering (8.4 ms), decoding (1.1 ms), and a fixed 12.5 ms display scan-out residual. The presentation term is set to $L_{\text{present}} = 0$ because `requestVideoFrameCallback` was not emitted, which biases the assembled estimate downward. Although

the lower-bound p_{95} of 70.9 ms is numerically below the 200 ms target, the omitted capture and presentation terms, directional asymmetry, and limited path diversity prevent any claim of measured headroom or capture-to-paint validation.

This estimate inherits the reduced-fidelity component-sum boundary stated canonically in §5.6.1: the synthetic camera and no-real-capture assembly mean it is not a capture-to-paint measurement, presentation is set to zero, scan-out is a fixed constant, and each one-way leg assumes directional symmetry when halving a round-trip time. The omitted sensor and interface capture delay is placed by the sensitivity model between tens of milliseconds and approximately 100 ms. In addition, both legs here share one laptop, Chromium process, and access network, so asymmetry across Internet service providers and network address translation paths is not exercised. Validation with a real camera, two distinct Internet service providers, and replication across paths and days remains future work (§5.6).

5.2.7 Multi-Hop Latency and Capacity Model

A second wide-area run estimated the network component on one relay-to-relay link by halving its round-trip time; it did not measure total forwarding latency. On the Malaysia West–Korea Central path, the implemented forwarding stack produced an estimated one-way network time of $t_{\text{hop,net}} = 34.5$ ms ($p_{50} = 34.51$ ms and $p_{95} = 35.00$ ms; $n = 116$ time-series samples from one link in one run; standard deviation 0.81 ms). These samples describe temporal variation on one link rather than 116 independent geographic paths, and the halving assumes directional symmetry. Relay processing was measured separately with a hermetic mediasoup Direct-Transport echo path: approximately 1.2 ms at p_{50} and 2.2 ms at p_{95} under 60-way fan-out, with a single-host p_{99} scheduling tail of approximately 9.5 ms. Combining representative p_{50} inputs gives the modelled quantity

$t_{\text{hop,total}} = t_{\text{hop,net}} + 1.2$ ms. This cross-campaign combination is a model input rather than a measured production total.

The sensitivity model uses $L_{\text{fixed}} = 25$ ms, a last-mile term of 39 ms, and $t_{\text{hop,net}} = 34.5$ ms in the cascade-tree worst-path expression

$$L_{\text{worst}} = L_{\text{fixed}} + L_{\text{last-mile}} + d\, t_{\text{hop,net}}.$$

At diameter $d = 0$, the formula reconstructs approximately 64 ms from two inputs derived from the first wide-area run, close to its 64.8 ms estimate. This is an arithmetic consistency check rather than independent validation. The $L_{\text{fixed}} = 25$ ms input is a synthetic-camera lower bound that excludes real capture. At the approximately 100 ms real-webcam sensitivity point, $L_{\text{fixed}} + L_{\text{last-mile}}$ rises from approximately 64 ms to approximately 139 ms and every $N_{\max}$ decreases. The projections are therefore optimistic with respect to capture latency. Table 5.5 reports the resulting viewer-count sensitivity within a one-way budget.

Table 5.5: Analytical cascade-tree latency sensitivity within a 300 ms one-way budget at the synthetic-camera lower bound $L_{\text{fixed}} = 25$ ms. Each row varies both the audio-forwarding regime and degree cap D. Values of $N_{\max}$ are model outputs rather than observed capacities. The separate relay-capacity experiment measured through $N = 15$ and supports an $N = 100$ projection (§5.4); it does not validate the values in this table.

Audio regime	**Degree cap** D	$N_{\max}$
Linear ($O(N)$ last-N audio)	2	≥ 200
Quadratic ($O(N^2)$ forward-all)	3	100
Quadratic ($O(N^2)$ forward-all)	4	150

This table is an *analytical sensitivity*, not a validated capacity result. Each row changes both the audio-forwarding regime and degree cap D,

so the spread in $N_{\max}$ cannot be attributed to either input alone. The implemented default forwards all audio and has $O(N^2)$ fan-out; an optional last-N gate reduces this towards $O(Nk)$ but is not yet cross-relay. Both $D = 3$ and $D = 4$ remain quadratic but produce different limits. Thus, $N_{\max} = 100$ is a conditional model output rather than an observed capacity or universal floor. The table also uses a synthetic-camera latency lower bound and hypothetical degree caps. Tree mode is disabled by default, and `deriveDegreeCap` is not connected to the runtime path. The $N_{\max} = 100$ latency-model output is distinct from the capacity projection in §5.4, which is based on measurements through $N = 15$. The 300 ms sensitivity budget likewise does not provide evidence for the thesis's sub-200 ms design target.

5.2.8 End-to-End Encryption Overhead

When transform attachment succeeds, DVConf's content-blind relay supports end-to-end encryption through a client-side SFrame `RTCRtpScriptTransform`; the current shim otherwise fails open. The transform adds a worker-thread transition and per-frame AES-GCM-256 processing that a cryptographic micro-benchmark of the AES operation alone cannot represent. A controlled two-arm loopback comparison ($n = 12$ per arm) measured the enabled-minus-disabled difference in the same one-way component-sum estimator (Table 5.6); it was not a direct camera-to-display or matched wide-area measurement.

The observed differences between arms are small relative to their session-level standard deviations (1.55 ms with E2EE disabled and 1.22 ms with E2EE enabled). Without an uncertainty or equivalence analysis, no non-zero overhead was resolved at $n = 12$ in each arm. The decryption worker attached successfully, and the consumer decoded frames through the content-blind SFU without silent drops. At this sample size, p_{95} and p_{99} are close to the maximum; the result therefore shows unresolved loopback overhead rather than zero overhead. The loopback

Table 5.6: E2EE-enabled and E2EE-disabled loopback component-sum estimates, $n = 12$ per arm. These are not direct capture-to-display measurements.

Arm	p_{50} **(ms)**	p_{95} **(ms)**	p_{99} **(ms)**
E2EE disabled	30.9	34.7	34.7
E2EE enabled	31.2	34.4	34.4
Δ (enabled minus disabled)	+0.3	−0.3	−0.3

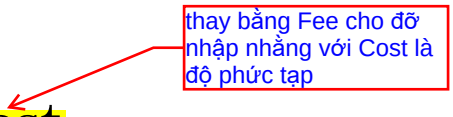
result stands at that stated scope; a subsequent matched cloud-WAN comparison with a larger sample per arm is reported below. The SFrame path is also opt-in and may fail open if transform attachment fails (§6.3); neither timing experiment alters that security limitation.

Enabling end-to-end encryption added a small but resolved mean one-way overhead on a matched cloud-WAN path: across $n = 30$ per-session medians in each arm, the block-stratified mean difference is +1.86 ms with a 95% stratified-bootstrap confidence interval of $[+1.14, +2.60]$ ms, and because the interval excludes zero, a small non-zero mean overhead is resolved in this window. The comparison was predeclared and matched: one daemon window executed six blocks, each containing five E2EE-disabled and five E2EE-enabled sessions, so that the two arms alternate through the window instead of running back-to-back (a counterbalanced design), with the producer client on an Azure Japan East virtual machine, the consumer client on a Korea Central virtual machine, and the relay, signalling, and pinned Sui localnet on a separate Korea Central host. The disabled arm gives $p_{50}/p_{95}/p_{99} = 36.64/38.36/40.76$ ms and the enabled arm 38.66/41.84/41.87 ms.

Several boundaries scope this result. Both arms use the same synthetic-camera one-way component-sum estimator as the wide-area campaign, with $L_{\text{present}} = 0$ and the fixed 12.5 ms scan-out residual, so the comparison is a difference of two lower-bound arms rather than a capture-to-paint mea-

surement. The tail is unresolved: the p_{95} difference of +3.48 ms has a confidence interval of $[-0.13, +4.12]$ ms, which includes zero, so no tail overhead is resolved. Both clients ran on Azure data-centre egress paths, not consumer access networks; consumer Internet-service-provider and network-address-translation diversity was explicitly deferred by this window and remains open. Attributing the mean difference to endpoint encrypt and decrypt work is an interpretation consistent with the content-blind relay design, not a measured attribution. Separately, the disabled arm is the first two-peer, distinct-region one-way estimate in this study; the §5.2.6 campaign placed both peers on one laptop. Fail-closed operation, explicit key management, rekey behaviour, and replication over consumer access networks remain future work (§5.6).

The provenance for this window is run `p2wan-20260716T065602Z` (2026-07-16), whose raw records and manifest are retained in the evaluation archive. The confidence intervals come from a stratified bootstrap with 10,000 iterations and seed 20260716. Per-block median differences are all positive, from +0.36 to +2.69 ms. Every enabled session evidences SFrame `RTCRtpScriptTransform` encrypt attachment on the producer leg and decrypt attachment on the consumer leg in the committed per-session logs.



5.3 Cost

5.3.1 Proposal Estimate and Evaluation Scope

The proposal reports a preliminary transaction-cost estimate of approximately **USD 0.01 for room creation**. It neither defines that value as an acceptance threshold nor extends it to a complete session. This section therefore does not use USD 0.01 as a pass–fail criterion. It reports measured `effects.gasUsed` values for implemented transactions, retains the earlier one-relay projection for comparison, and reports an executed

complete session comprising room creation, escrow, pairing assignment, the $K \times N = 8$ validator proof submissions, closure, and reward distribution. The contract design determines which operations and configuration-dependent proof counts are incurred, while the resulting USD amount also depends on the protocol gas schedule, reference gas price, and SUI price. At the stated assumption of USD 1.50 per SUI, computation for one call is approximately 1.09 million MIST, or USD 0.0016 (§5.3.3).

5.3.2 Methodology

Sui gas has two layers. **Gas units** comprise computation buckets and storage bytes written, less a storage rebate; the protocol configuration is keyed by `protocol_version`. The **MIST cost** equals gas units multiplied by the reference gas price. The complete-session run used framework revision `94ad8ccd0ed6`, contracts revision `17e1fce`, protocol version 113, reference gas price 1000, and Sui CLI 1.66.2. The raw record also discloses dirty workspace, daemon, and contract-source fingerprints; the result is provenance-bounded rather than a clean-checkout byte reproduction. This pinned localnet supports a direct local gas claim, but not an absolute mainnet MIST claim (§5.1.5, §5.6).

The harness starts a local validator, publishes `dvconf-contracts`, and drives the node-enrolment and room lifecycle with the daemons' transaction builders. These comprise the dual-key Ed25519 session-proof builder, reward-distribution trigger, and heartbeat, registration, and voting builders. All four `gasUsed` fields are logged for each transaction. A deterministic aggregator reads the raw JSONL, produces the tables below, and records an output SHA-256 digest, making the derivation byte-reproducible. A two-run cross-check produced identical computation gas for the ten operations present in both runs. Storage gas varied with the size of the mutated object. The section therefore treats computation as fixed for the pinned protocol and reports storage for one representative ses-

sion trajectory. This method supersedes the earlier Move Virtual Machine instruction proxy, which excluded native cryptography, event emission, object-storage gas, and programmable-transaction-block overhead. The dominant `submit_session_proof` term is consequently measured directly rather than reconstructed from the gas schedule.

5.3.3 Measured Per-Function Gas

Two measures are reported. **Irreversible cost** equals computation plus the non-refundable storage fee and provides a lower bound on cost that cannot be recovered. **Net cost** equals computation plus storage charges less the rebate; it includes storage deposits that may later be returned as objects are deleted. Table 5.7 lists the functions relevant to a session.

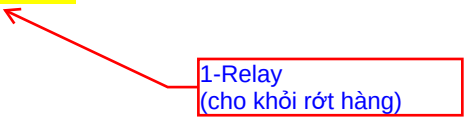
Table 5.7: Measured per-function gas for session transactions (millions of MIST; irreversible cost equals computation plus the non-refundable storage fee). Package publication and node enrolment are one-time costs amortised across sessions.

Function	**Computation**	**Irreversible**
`create_room`	1.090	1.161
`create_escrow`	1.090	1.100
`submit_pairing_proposal`	1.140	1.275
`submit_session_proof` (per validator)	**1.160**	**1.327**
`close_room`	1.090	1.173
`distribute_rewards`	1.150	1.367

Three one-time classes were also measured. Package publication incurred 5.18 million irreversible MIST, equivalent to approximately USD 0.00777 at USD 1.50 per SUI, plus an approximately 0.55-SUI refundable storage deposit. Per-user registration incurred 1.22 million MIST, or approximately USD 0.00182. A synthetic six-operation enrolment-coverage bundle incurred 7.18 million MIST, or approximately

USD 0.01077. Because relay and validator roles are mutually exclusive, this bundle is coverage evidence rather than a realisable per-node cost. The evaluation record contains all 20 measured functions.

5.3.4 Historical One-Relay Projection by Configuration



The earlier comparison model uses

$$C_{\text{session}}(N) = C_{\text{fixed}} + NC_{\text{proof}},$$

where C_{fixed} comprises room creation, escrow creation, pairing, closure, and reward distribution, while C_{proof} is the measured session-proof cost. One-time deployment, enrolment, and optional heartbeat costs are excluded. Project-specific ADR-0006 documents a four-validator design with a three-of-four quorum. As established in Chapter 1, ADRs are traceability artefacts for project decisions rather than independent academic evidence. The implemented settlement path accepts two proofs and computes their arithmetic mean; this cost projection therefore does not validate the designed quorum. The N-scaling term uses approximately 1.327 million irreversible MIST per validator proof. The historical model includes one assigned relay ($K = 1$), whereas the implemented default assigns a primary and standby ($K = 2$) and submits one proof per validator for each relay. Table 5.8 retains the $K = 1$ lower-bound projection for sensitivity comparison; the direct $K = 2$ result in §5.3.5 supersedes it as the implemented-default session result. The USD columns use the stated assumption of USD 1.50 per SUI.

Two categories sit outside the core. `RelayHeartbeat` is implemented and measured at approximately 1.09 million MIST of computation per emission; the estimate of ten emissions per second for 100 relays is a throughput projection rather than a load test. Three signalling-penalty

Table 5.8: Projected $K = 1$ per-session cost by validator configuration. USD values use the stated assumption of USD 1.50 per SUI.

N	**Irrev. cost (SUI)**	**Irrev. cost (USD)**	**Net cost (USD)**
2	0.008729	0.0131	0.0304
3	0.010056	0.0151	0.0313
4	**0.011383**	**0.0171**	**0.0322**
5	0.012709	0.0191	0.0330

elements from the original projection—`submit_equivocation_proof`, `SlashVoteBox`, and `cast_slash_vote`—are absent from the implementation and remain proxy estimates. They are distinct from the implemented relay-quality path, in which `economic_layer::pay_slash` performs a later cooperative deduction; that transaction was not costed here.

5.3.5 Measured Two-Relay Full-Session Cost

On localnet protocol 113 at reference gas price 1000 MIST, the executed $K = 2$, $N = 4$ trajectory comprised 13 successful transactions: eight actual proof submissions and five fixed lifecycle calls. The retained raw JSONL has SHA-256 `a3892854...3b3cd1`; the provenance ledger retains the complete digest.

Table 5.9: Measured pinned-localnet $K = 2$, $N = 4$ full-session trajectory. Corresponding USD values are reported below under the labelled scenario of USD 1.50 per SUI; they are not a market-price or mainnet measurement.

Scope	**Txs**	**Computation (MIST)**	**Irreversible (MIST / SUI)**	**Net (MIST / SUI)**
$K = 2$, $N = 4$	13	14,930,000	17,134,988 / 0.017134988	35,048,188 / 0.035048188

At the labelled USD 1.50-per-SUI scenario, the measured trajectory is

USD 0.025702 irreversible and USD 0.052572 net. The proposal's preliminary USD 0.01 figure concerns room creation only, so it is not a like-for-like acceptance threshold. These are pinned-localnet transaction effects, not mainnet cost or validation of the designed three-of-four settlement rule.

For comparison, the fixed lifecycle contributes 6.08 million irreversible MIST in the historical model, while each validator proof adds approximately 1.33 million. Increasing N from 2 to 4 therefore doubles that model's proof sub-term but increases its projected $K = 1$ full-session irreversible cost by approximately 30%, from 8.73 to 11.38 million MIST. The direct $K = 2$ result removes the relay-count extrapolation, but remains cost evidence rather than proof of Byzantine quorum enforcement. Aggregate signatures are the principal architectural optimisation considered in §5.3.6.

5.3.6 Correction to the BFT Baseline and Aggregation

ADR-0006 stated that the earlier configuration used $n = 3$. Source inspection instead finds that `DEFAULT_MIN_VALIDATORS_PER_ROOM` was 2 and rose to 3 only for rooms with 9–11 participants. Moving the default to $N = 4$ therefore doubles the proof sub-term for a six-participant room rather than increasing it by 33%. The evaluated build sets the default to 4 and `QUORUM_THRESHOLD` to 3. This is a design and configuration result, not evidence that settlement enforces three-of-four agreement. Its cost implication is $K \times N$ proof submissions under the implemented $K = 2$ assignment default. BLS12-381 aggregate signatures could remove the linear verification term, but were not implemented.

Three cost limitations carry to §5.6. First, the three signalling-penalty terms are absent from the implementation and remain projections. Second, mainnet uses a different reference gas price and may use a different gas schedule, so no exact mainnet cost is inferred from the localnet mea-

surement. Third, storage charges are state-dependent and are reported for one representative trajectory.

5.4 Capacity

5.4.1 Proposal Scale and the Two-Layer Split

The proposal reports peak concurrent calls with **5–10 participants across multiple rooms** and sets a Phase 2 objective of testing beyond that scale; it does not define a 5–10-participant per-room target. Two independent subsystems determine the evaluated capacity. The **signalling** layer (`apps/signaling`) is constrained by WebSocket connections and quadratic offer, answer, and ICE fan-out. The **media** layer (`apps/relay`) is constrained by transports, producers, consumers, CPU, and egress bandwidth. Refusal of an eleventh WebSocket from the same source IP address is therefore an anti-abuse result, whereas media saturation would be a hardware result. The signalling control is exercised by an in-process protocol driver (§5.4.2); the media layer is profiled on multi-host infrastructure through $N = 15$, without observing an operating ceiling (§5.4.3). Operation with hundreds of participants remains outside the measured scope.

5.4.2 Signalling Capacity and the Per-IP Limit

The protocol driver connects synthetic peers and exercises the implemented rate controls without chain bootstrap. The retained script is `signaling-stress.ts`; it imports the production `createServer()` function. Single-host 30-second arms completed S1 at $N = \{2, 4, 6, 8, 10\}$ and S3 at $M = \{1, 2, 4\}$ rooms with $P = 4$ peers per room. Every accepted arm had zero message drops; S1 accepted every peer through $N = 10$, and S3 accepted 4/4 and 8/8 before the one-source 4×4 arm reached the per-source shield (Table 5.10).

Table 5.10: Local signalling matrix conducted on 2026-07-15. Each point is one 30-second, single-host trial. Drops count messages among accepted peers; loopback source aliases are a controlled source-key experiment, not independent devices or access networks.

Arm	**Requested source keys**	**Accepted**	**Drops**	**Fan-out $p_{50}/p_{95}/p_{99}$ (ms)**
S1 sweep, $N = 2, 4, 6, 8, 10$	One loopback source	All peers at every point	0	At most 2 ms at reported $N \geq 4$ points
S3, $M = 1$, $P = 4$	One loopback source	4/4	0	0/1/1
S3, $M = 2$, $P = 4$	One loopback source	8/8	0	1/3/3
S3, $M = 4$, $P = 4$	One loopback source	10/16	0	1/2/2
S3, $M = 4$, $P = 4$	Four requested `127.0.0.x` sources	16/16	0	1/2/2

The fixed 4×4 comparison isolates the source-key boundary. With one loopback source, the server accepted 10/16 peers and rejected six with WebSocket close code 4029 at `MAX_CONNECTIONS_PER_IP=10`; round-robin binding of the same workload to four requested `127.0.0.x` source addresses accepted 16/16 with zero drops. This closes the controlled local source-key matrix, but loopback aliases do not establish independent-device, NAT, autonomous-system, or Internet-service-provider capacity. The configured ten-connection shield is an anti-abuse boundary, not a media-capacity ceiling. The media layer is evaluated separately through $N = 15$. The implementation also limits each connection to 100 messages per second and a 64 KB payload.

5.4.3 Media-Layer Capacity: The Internet Campaign

The media layer was profiled by a multi-host internet saturation campaign (2026-07-09 plaintext baseline, 2026-07-10 E2EE arm and co-location control) over a Korea–Japan two-relay topology with a separate synthetic-consumer host. The campaign exercised a single-room viewer ramp through $N = 15$ without reaching a relay operating ceiling. The $N = 100$ figure below is therefore modelled from measured low-load slopes and configured inputs; it is not a measured ceiling. Three findings follow.

Relay CPU remained below saturation in the measured range. For the single-room viewer ramp, `cpuCoresSrtp` was 0.023, 0.040, and 0.050 of one core at $N = 5$, 10, and 15, respectively, in the E2EE arm. The run therefore records `binding=not-saturated`. Inverting the measured per-path CPU slope gives a headroom indicator of approximately 2,250 forwarding paths per worker. This slope-derived indicator is not an operating ceiling because the relay peaked at only 0.05 of one core.

Client-side tail growth is consistent with decode contention. Glass-to-glass p_{99} increased from 96.0 to 166.8 ms in the plaintext arm and from 118.3 to 172.3 ms in the E2EE arm between $N = 10$ and $N = 15$. All synthetic consumers ran on one two-vCPU host, so the increase is consistent with shared decode-CPU contention. In the co-location control, halving decode density approximately halved L_{decode} while relay `cpuCoresSrtp` remained stable. The isolated L_{decode} term therefore reflects contention downstream of the relay. Attribution of the entire glass-to-glass p_{99} increase remains inferential because the reported median did not recede in the three-trial control; its residual tail followed a cross-region jitter-buffer transient. The projection in §5.4.4 does not depend on this client-side tail.

No material relay-capacity difference was resolved through $N = 15$ between E2EE and plaintext. Because the relay forwards ciphertext that it cannot decode, SFrame operates as a client-side transform. Forwarding CPU and per-viewer egress, approximately 2.92 Mbps, showed

no material difference between the encryption-disabled and encryption-enabled arms at $N = 5$, 10, and 15. These are comparable-load runs conducted on different days and bench commits with mirrored placement, rather than matched experimental arms. The values are close but not identical: for example, `cpuCoresSrtp` was 0.0525 and 0.0500 at $N = 15$, and per-viewer egress differed at each rung. No repeated-arm equivalence test was performed. The 22 ms cross-run difference in glass-to-glass p_{99} at $N = 10$ is consistent with consumer-endpoint decode work on the co-located host, rather than a resolved relay cost. This is the capacity-side counterpart of the loopback E2EE result in §5.2.8.

5.4.4 The 100-User Relay-Count Model

Let R_{CPU} and R_{BW} denote the relay counts required by the CPU and bandwidth ceilings, respectively. The projected count is

$$R = \max(R_{\mathrm{CPU}}, R_{\mathrm{BW}}) .$$

A single-worker `DirectTransport` benchmark gives $C_{\mathrm{worker}} \approx 540$ forwarding paths per worker, while the wide-area SRTP run indicates approximately 2,250 paths of headroom. Because the DirectTransport benchmark omits SRTP and uses one host, 540 is an optimistic calibration point rather than a production ceiling. The implemented planning default is $C_{\mathrm{worker}} = 300$. With nine forwarded paths per viewer and measured egress of approximately 2.92 Mbps per viewer, Table 5.11 gives the projected 100-user relay count.

The configured bandwidth ceiling sets a three-relay floor under the reported inputs. The model uses a configured 100-Mbps-per-relay egress limit rather than a measured physical network-interface ceiling. Across the reported CPU inputs, the 100-user projection has a floor of three relays. At the implemented $C_{\mathrm{worker}} = 300$ planning value, CPU produces the same floor; a lower production value would make CPU the binding

Table 5.11: Projected relay count for 100 users under independent CPU and bandwidth ceilings. Across the reported CPU inputs, the configured 100-Mbps-per-relay egress limit gives a floor of three relays. At the implemented planning value $C_{\text{worker}} = 300$, the CPU term also equals three. The projection is based on measurements through $N = 15$ and is not a measured 100-user run.

Axis	**Formula** ($N = 100$)	**Relays**
CPU forwarding	$\lceil 900 \text{ paths} \div 540 \rceil$ (workstation calibration)	2 (1 at 2,250)
Egress bandwidth	$\lceil \approx 290 \text{ Mbps} \div 100 \text{ Mbps} \rceil$ (registered limit)	3

term. Applying burst and utilisation allowances gives a planning estimate of approximately five to six relays. Peak measured egress was 43.9 Mbps at $N = 15$, so the physical network interface did not constrain the experiment. These values remain projections for $N = 100$, not observations of a 100-user room.

Two limitations carry to §5.6. The local signalling matrix distinguishes the per-source shield from the fixed multi-room workload, but real multi-device, NAT, autonomous-system, and Internet-service-provider replication remains future work. Participant-driven automatic scaling is also not implemented: the control-plane handler sets `expectedParticipants=0`, and `deriveDegreeCap` is used only in tests rather than the runtime path. The mesh path for $R \geq 3$ was exercised separately.

5.5 Failover

5.5.1 Proposal Objective and the Two Recovery Designs

The proposal calls for self-healing routing under localised outages. This thesis operationalises that objective as recovery from primary-relay failure

during a session. ADR-0004 records recovery-time design ranges of 8–15 s for SFU rooms and 10–18 s for MCU-composite rooms. ADR-0009 records a separate approximately 100 ms target for a warm static-mesh path. These Architecture Decision Records document accepted project choices; the full recovery intervals were not measured, but a narrower in-process mechanism floor was.

A separate in-process real-mediasoup warm-pipe bench completed 30 cutovers with zero failures and measured detection plus resume at $p_{50}/p_{95}/p_{99} = 58/74/80$ ms. It uses a replicated 50 ms RTP-silence watcher, two real Workers, and a pre-created paused standby `DirectTransport`, but models primary failure by closing the primary client sink while the publisher and pipe remain active. It is therefore an optimistic mechanism floor, not full failover MTTR; its exclusions are enumerated in §5.6.3.

Separately, the evaluation establishes permissionless on-chain promotion, offline detector transitions, and a live run in which promotion occurred after verified pre-failure media. Post-promotion browser-rendered continuity, complete client recovery, and end-to-end recovery time remain unmeasured.

5.5.2 Implemented Promotion Mechanism

Recovery is actuated on-chain by `promote_relay`, which emits `RelayPromoted`. The entry is permissionless: it accepts neither `AdminCap` nor a capability object. Instead, the contract requires the primary heartbeat to be more than three epochs old and the candidate to be among the room's assigned relays. The staleness threshold prevents early displacement of a healthy primary. Candidate freshness is not contract-enforced, however; once the primary is stale, a caller may nominate any assigned relay, while a correctly operating control-plane watcher ranks candidates using off-chain freshness data. This mechanism supersedes the administrative `swap_relay` path retained as legacy infrastructure.

5.5.3 Promotion Trigger and Legacy Detector

`RelayHeartbeatWatcher` is the sole implemented automatic trigger in the promotion path. It submits `promote_relay` when the primary heartbeat satisfies the epoch-based staleness predicate. A separate legacy analyser, imported only by tests and the smoke-test harness, models triggers for penalties, missed heartbeats, and degraded performance in the superseded `emit_swap_relay` path. Its approximately 30 s missed-heartbeat window is a legacy design parameter, not a measured recovery bound. The trigger codes distinguish a `RelaySlashed` event, a `RelayPerformanceDegraded` warning, and three consecutive missed heartbeats at ten-second intervals. Shortening the heartbeat interval would reduce the design detection window while increasing the chain-event rate.

The legacy detector is a pure function from normalised chain events and timer ticks to decision records. Its state machine enters `TRIGGERED` on a fault, requests standby replacement when appropriate, and enters `TRIGGERED_NO_STANDBY` on bilateral failure. It resets the missed-heartbeat counter when a fresh primary heartbeat arrives and emits one decision per trigger. Eleven unit tests and six fixed smoke-test scenarios passed during evaluation. This evidence exercises the expected transition matrix; it does not time the transitions or establish the behaviour of the implemented `promote_relay` watcher. The related Move suite passed 386 tests at the pinned contracts revision; the earlier 382-test count was a 2026-07-13 historical snapshot.

5.5.4 Relay-Termination Experiment

The mechanism was exercised in a three-relay localnet run on 2026-07-08. Four peers produced media before the primary process was terminated, and an independent Sui RPC query subsequently reported a new primary at epoch 6. A room-assignment query confirmed that the promoted relay occupied the primary slot, and the surviving relay processes remained open.

A second termination produced another promotion at epoch 10. These observations establish that promotion occurred after verified pre-failure media and could occur sequentially. They do not establish post-promotion media continuity or a full recovery interval; a subsequent instrumented decomposition (§5.5.5) measured only the on-chain submission-to-`RelayPromoted` component.

One limitation carries to §5.6: after termination of one of three relays, `assigned_relays` retains three positional slots but contains only two distinct operational relays. Restoring three distinct relays requires runtime assignment growth, which was outside the experiment. The run therefore establishes degradation without migration rather than sustained three-relay coverage.

5.5.5 Recovery-Time Budget, Forensics, and Scope

ADR-0004 associates the 8–15 s SFU target with four sequential phase estimates: detection and decision (1–10 s), promotion (sub-second by design), relay ramp-up (4–7 s), and client reconnection with re-issued TURN credentials (3–5 s). Summing the stated endpoints yields approximately 8 s to less than 23 s, so these estimates do not arithmetically substantiate the stated 8–15 s range. The detector tests exercise state transitions, but do not measure elapsed detection-and-decision time. Of the four phases, only the promotion term is now instrumented: across three localnet runs the interval from `promote_relay` submission to the matching `RelayPromoted` consensus commit was approximately 1.6 to 2.0 s (median 1.6 s), a pinned-localnet consensus-commit floor of the same order as, though modestly above, the sub-second design estimate. This localnet consensus-commit latency is not equivalent to client-observed event visibility, so no sub-second application-level recovery result is inferred. Detection and decision, relay ramp-up, and client reconnection remain unmeasured, and the 58/74/80 ms warm-pipe result omits these phases rather than replacing them.

The same runs bracket the preceding interval from primary process kill to promotion submission at approximately 225 to 231 s. This figure is a configuration artefact rather than a fault-detection measurement: the contract gates displacement on the primary heartbeat exceeding three epochs, and the localnet uses 60 s epochs, so the roughly 3.8-epoch wait is arithmetic. Under mainnet epoch durations, on the order of twenty-four hours, the same predicate would defer eligible displacement by days; the minutes-scale figure is specific to the 60 s-epoch localnet and is not a recovery or mean-time-to-recovery result.

Every claim about on-chain promotion and assignment is anchored by events re-read through RPC; media liveness is supported by process and socket telemetry. The permissionless `RelayPromoted` event records the room, previous primary, new primary, and epoch, but not the initiating trigger. Trigger and cascade markers occur only in the legacy `RelayFailoverInitiated` event. A promotion transcript can therefore reconstruct promotion and assignment history, but not trigger provenance. DVConf forensics are **signature-based rather than video-based**. The forensic event schema permits a future reporting command to reconstruct a room's promotion and assignment history offline.

5.6 Threats to Validity

This section records the evidential limits of the principal quantitative claims in §5.2 through §5.5. Each threat identifies the affected dimension, likely direction of bias, and an existing or proposed mitigation.

5.6.1 Single-Host and Instrumentation Bias

The localhost media-latency baseline, local signalling RTT run, local signalling capacity matrix, chain-latency campaign, cost measurement, failover analyser, and warm-pipe bench use one workstation. This biases

local latency and mechanism-floor timing downward and signalling capacity upward, except where the per-source connection limit constrains the test. The chain samples additionally share a persistent localnet and warm state, and their observer timing includes the configured 5 s polling cadence. By contrast, the wide-area latency and media-capacity campaigns used Azure hosts. The single-host caveat therefore applies only to the named local experiments, not to the network-inclusive observations. No reported latency figure constitutes a production service-level agreement. The reduced-fidelity component-sum latency figures — the wide-area campaign (§5.2.6), the localhost loopback proxy (§5.2.5), and both arms of the E2EE comparison (§5.2.8) — are assembled from WebRTC statistics with a synthetic camera and no real capture device, so none is calibrated against a physical capture-to-paint measurement; each also derives one-way delay from round trips and treats the unobserved presentation term as zero. This is the canonical statement of that reduced-fidelity boundary, which those sections cross-reference rather than restate. The net error of the Option-B estimator and the observer effect of 1 Hz statistics polling also remain uncalibrated.

5.6.2 Cost and Capacity Threats

Cost measured on a pinned localnet. Section 5.3 reports per-function `effects.gasUsed` and one complete $K = 2$, $N = 4$ trajectory from a pinned localnet. Absolute mainnet MIST requires a mainnet cross-check because the gas schedule, storage charges, and reference gas price may differ. The proposal's approximately USD 0.01 statement concerns room creation and is treated as a preliminary estimate, not as a full-session objective. At the labelled USD 1.50 per SUI, the measured localnet trajectory is USD 0.025702 irreversible and USD 0.052572 net. Three signalling-penalty terms are absent from the implementation and remain projected, and no batching or cross-transaction amortisation is evaluated.

Capacity under optimistic single-host forwarding. The media calibration uses `DirectTransport` without SRTP, so $C_{\text{worker}} \approx 540$ is optimistic for a production relay. The 100-user projection has a floor of three relays under the reported CPU inputs and configured bandwidth input, with a planning value of approximately five to six relays. At the implemented $C_{\text{worker}} = 300$ planning value, CPU and bandwidth both produce a floor of three; a lower production value would make CPU binding. The signalling matrix uses synthetic protocol peers and requested loopback aliases rather than real browsers, encoding, RTP, ICE, devices, NATs, or Internet service providers. Media measurements extend through $N = 15$; $N = 100$ remains a projection.

5.6.3 Failover Threats

Narrow timed floor plus boolean live evidence. The in-process real-mediasoup warm-pipe bench (§5.5.1) times a 58/74/80 ms mechanism floor by modelling primary failure as closing the primary client sink while the publisher and pipe stay active, and it excludes relay-process kill, on-chain promotion, browser jitter and rendering, and WAN delay, so it is an optimistic mechanism floor and not a failover mean-time-to-recovery; this is the canonical statement of that floor, which the failover sections cross-reference rather than restate. The live run shows that promotion occurred but does not time resumed media. No run measures the complete mediasoup pipe handshake after process loss, ffmpeg restart, client ICE renegotiation, or post-promotion browser-rendered continuity. The promotion-specific submission-to-`RelayPromoted` consensus-commit interval was, however, instrumented across three localnet runs at approximately 1.6 to 2.0 s (median 1.6 s), a pinned-localnet floor that is neither client-visible visibility nor a full recovery time. Full end-to-end recovery targets therefore remain design targets rather than measured results.

Promotion authority. The implemented `promote_relay` entry is

permissionless and accepts no `AdminCap`; its ordered guards culminate in the primary-staleness check (§5.5.2). The legacy `swap_relay` function remains an administrative fallback rather than the liveness actuator. The offline detector recognises bilateral failure through `TRIGGERED_NO_STANDBY`, but user-visible termination was not exercised. The second observed promotion showed sequential actuation with a surviving relay, not the no-standby termination state.

5.6.4 Evidence Supporting Each Claim

The failover smoke tests report deterministic decision-matrix outcomes and do not support statistical generalisation. Each signalling-matrix point is one 30-second trial; its timing and resource values are not population estimates. The warm-pipe bench contains 30 cutovers within one in-process topology, the wide-area latency run contains $n = 30$ sessions, the loopback E2EE comparison contains $n = 12$ sessions in each arm, and the matched cloud-WAN E2EE window contains $n = 30$ sessions in each arm on Azure data-centre paths. The chain campaign contains 30 sequential transactions per metric on one persistent localnet; these are shared-state observations rather than independent sessions. Table 5.12 maps each principal claim to its evidence and remaining validation requirement.

Table 5.12: Evidence audit by claim. Each disposition distinguishes direct measurement, projection, lower-bound estimation, and design targets.

Claim	**Evidence**	**Disposition**	**Required validation**
Sub-200 ms glass-to-glass latency	§5.2 wide-area run ($n = 30$)	One-way $p_{95} = 70.9$ ms reduced-fidelity lower-bound sum; not a capture-to-paint measurement	Real camera and two distinct Internet service providers
E2EE latency overhead	§5.2.8 loopback ($n = 12$ per arm) and matched cloud-WAN window ($n = 30$ per arm, data-centre paths)	Loopback difference unresolved; cloud-WAN mean +1.86 ms one-way overhead resolved, 95% CI $[+1.14, +2.60]$ ms; tail (p_{95}) difference unresolved	Consumer-ISP-path and rekey experiment; fail-closed operation
Full-session on-chain cost	§5.3 executed pinned-localnet $K = 2$, $N = 4$ trajectory	13 transactions; USD 0.025702 irreversible and USD 0.052572 net at labelled USD 1.50 per SUI	Mainnet or comparable-public-network replication

Continued on the next page

Table 5.12 (continued)

Claim	Evidence	Disposition	Required validation
On-chain create/settle observer latency	§5.2.4 pinned localnet, $n = 30$ per metric	SDK call to exact production-poller event: create 5003/5018/5020 ms and settle 4743/5120/5184 ms at $p_{50}/p_{95}/p_{99}$; includes 5 s polling	Public-network replication plus browser, join, promotion, and recovery timing
Signalling RTT floor	§5.2 loopback: eight sockets, $n = 48$	Same-process $p_{50}/p_{95}/p_{99} =$ 1/3/3 ms; nested, not independent	Cross-host, WAN, and independent-run replication
Signalling source-key boundary	§5.4 local S1/S3 matrix	Fixed 4×4: one source 10/16 versus four requested loopback sources 16/16; zero drops among accepted peers	Real multi-device, NAT, ASN, and ISP replication
Media-forwarding calibration C_{worker}	§5.4 saturation benchmark	540 forwarding paths per worker under `DirectTransport`	SRTP forwarding and multi-host relay calibration

Continued on the next page

Table 5.12 (continued)

Claim	Evidence	Disposition	Required validation
100-user relay-count projection	§5.4 two-ceiling model	Floor of three and planning value of five to six, projected from measurements through $N = 15$	Operational $N = 100$ run and participant-driven scaling
Relay-to-client path RTT	§5.2 `L_relay_fwd` trace ($n = 118$)	Direct path-specific RTT: $p_{50} = 96.71$, $p_{95} = 117.49$, and $p_{99} = 296.71$ ms; not one-way or in-router processing	Replication across endpoints, paths, and days
Per-hop delay dominated by the network term	§5.2 relay-processing benchmark	Approximately 1.2 ms p_{50} processing proxy versus a 34.5 ms hop estimate; no production bound	SRTP forwarding path, including scheduling tails
Legacy failover-detector transitions	§5.5.3 detector and Move	Six smoke scenarios and eleven unit tests passed; the on-chain `u8` trigger-code field has no enumeration guard	Standby-bound and idempotence tests; enumeration validation

Continued on the next page

Table 5.12 (continued)

Claim	Evidence	Disposition	Required validation
Warm-pipe mechanism floor	§5.5.1 in-process real-mediasoup bench (30 cutovers)	Zero failures; detection plus resume $p_{50}/p_{95}/p_{99}$ = 58/74/80 ms; optimistic sink-close floor	Process kill, promotion, and browser-rendered recovery over WAN
Promotion is permissionless and staleness-gated	§5.5.2 and instrumented evaluation runs	Source inspection and observed promotion after pre-failure media; submission-to-`RelayPromoted` consensus-commit component measured at approximately 1.6–2.0 s (localnet floor); subsequent rendered continuity unmeasured	Full timed recovery (MTTR) and post-promotion continuity
Forensics are signature-based rather than video-based	§5.5.5 and CLI	CLI implemented; smoke test passed	Dedicated failover report

Continued on the next page

Table 5.12 (continued)

Claim	Evidence	Disposition	Required validation
Recovery within the 8–15 s SFU target	§5.5.5 phase estimates	Design target; endpoints span approximately 8 s to less than 23 s. The warm-pipe floor omits full recovery phases and client reattachment.	End-to-end process-kill-to-render experiment

The chapter's evidence resolves into one verdict per evaluation dimension. Latency evidence is direct for the relay-to-client path RTT and the pinned-localnet create-and-settle observer distributions. The sub-200 ms target itself is supported only by a one-way $p_{95} = 70.9$ ms lower-bound component sum rather than by capture-to-paint measurement, the per-hop comparison combines a hermetic processing proxy with a one-link network estimate, and public-network chain observation remains unresolved. Cost evidence is direct for per-function localnet gas and the executed $K = 2$, $N = 4$ session, and it remains a pinned-localnet result pending a mainnet cross-check. Capacity evidence is direct for the local signalling capacity matrix, while the 100-user relay count remains a projection based on measurements through $N = 15$. Failover evidence is direct for the in-process warm-pipe floor, the legacy failover-detector decision matrix, observed permissionless promotion with its measured submission-to-`RelayPromoted` consensus-commit component, and signature-based forensic reconstruction, while the full process-kill-to-rendered-media recovery time (MTTR) remains unresolved. Across dimensions, the local signalling RTT mechanism floor and the C_{worker} calibration point are likewise direct single-host measurements, and the matched cloud-WAN E2EE window resolves a +1.86 ms

mean one-way overhead ($n = 30$ per arm; 95% CI $[+1.14, +2.60]$ ms) on data-centre paths, with its tail (p_{95}) difference and consumer-access-network behaviour unresolved. Chapter 6 therefore treats the remaining items as explicit limitations and follow-up experiments rather than completed validation.

Chapter 6

Discussion and Limitations

6.1 Implemented System versus Proposal

Chapter 4 describes the implemented system, whereas the approved proposal (2026-04-03) defines the original commitments. This section compares the proposal's eight-component architecture and five-step data flow with the implementation. The comparison distinguishes completed, reduced-scope, reframed, and open elements; Table 6.1 assigns one primary status to each proposed component.

không thấy bảng 6.1

Items 1, 3, 5, and 8 have reduced scope; items 2 and 4 are reframed; item 6 is out of scope; and item 7 is deferred. Chapter 4 also introduces two components that were not enumerated as first-class entries in the proposal's eight-component architecture: **validator daemons**, which submit dual-signed composite-probe attestations, and the **economic layer**, which centralises reward and scarcity policy. Both are mentioned elsewhere in the proposal. The zero-quality settlement branch records a penalty obligation and emits `RelaySlashed`. Deduction occurs only through a later `pay_slash` transaction that requires the relay-owned `StakePosition`; the media-canary audit route likewise demonstrates co-operative self-penalisation. Section 6.3 consolidates the resulting evidence, custody, and quorum limitations.

The proposal's five-step flow maps to the implemented seven-phase life-

cycle (§3.5), with a $K = 2$ primary-and-standby static mesh rather than multi-mixer path distribution. Integration was exercised at bounded prototype depth, but not across all trust and recovery boundaries. The wide-area result ($n = 30$, one-way $p_{95} = 70.9$ ms) is a lower-bound component sum derived from half-round-trip estimates, not a capture-to-paint measurement (§5.2). Three-layer simulcast is implemented; estimator-driven selection and SVC remain outside the evaluated scope.

6.2 Scope Reductions and Future Work

The deferred work is not homogeneous. Table 6.2 groups it by theme and status: open, partial, mixed, or reframed.

Multi-relay delivery is partial: the implemented $K = 2$ static star and reverse-PipeTransport mesh forwards media in at most two hops, while cascade-tree boot integration and re-parenting remain open. The projected $N = 100$ envelope, which gives a planning estimate of approximately five to six relays under the stated inputs (§5.4), assumes that the mesh can be extended to that load. It is a planning model rather than measured capacity or evidence for the unimplemented tree orchestration.

6.2.1 TURN Deployment

The TURN and STUN work is **partial**: the design and credential-issuance code are implemented, but no operational coturn deployment was evaluated. Source inspection confirms the on-chain event surface and capability-gated entries in `turn_credential.move`; the two-secret overlap window recorded in ADR-0010; the control-plane HMAC-SHA1 issuer with 24-hour rotation and a `RelaySlashed` rejection hook; the `POST/turn/issue` endpoint; the relay-side fetcher and TURN-URL derivation; and the client-side merge of `iceServers` into mediasoup-client transport options.

The evaluation does not establish end-to-end chain-anchored encrypted-

secret distribution or deployment of coturn on the relay virtual machine. The test suites exercise the issuer path, but no coturn binary verified the HMAC against WebRTC traffic. Early revocation of `RoomCapability` and emergency secret rotation are separate controls: neither cryptographically revokes an issued TURN HMAC credential at coturn, which otherwise remains valid until expiry.

6.2.2 Reconciliation of Earlier Commitments

The interim progress report identified several planned or incomplete results. Table 6.3 reconciles each with the final state. Section 6.3 records the distinction between the three-of-four validator design and the implemented two-proof arithmetic mean, together with the remaining custody, identity, and recovery limitations; Section 6.2.1 records the separate coturn deployment gap.

6.3 Trust-Model Limitations

Table 6.4 derives the trust bounds from the implemented system. The principal limitations concern proof-to-relay binding, penalty custody, the validator quorum that is designed but not connected to settlement, score-only pairing, partial `AdminCap` migration, and the absence of an automatic clock-skew detector.

không thấy bảng 6.3

Table 6.4: Trust-model limitations derived from the implemented system.

Limitation	**Threat and implemented bound**	**Future-work direction**
Sui-network compromise	A fork, censorship, or rollback may invalidate the chain-anchored properties in §3.4. The thesis assumes a public chain with finality; substrate defence is outside scope.	Evaluate multi-substrate anchoring.
Proof-to-relay binding	The signed probe combines direct STUN observations with relay-reported counters and does not independently verify media forwarding. Submission authenticates a validator and registered relay, but does not require that relay to be assigned to the room.	Enforce assignment membership and use independently observed media-path evidence.
Quorum and penalty custody	Settlement requires two proofs and computes their arithmetic mean; it does not enforce the designed three-of-four rule. `RelaySlashed` records an obligation, while deduction requires a later relay-authorised `pay_slash` transaction. Room-lifetime stake locking is absent.	Enforce the quorum rule, protocol-controlled penalties, and synchronised stake locking.

Continued on the next page

Table 6.4 (continued)

Limitation	Threat and implemented bound	Future-work direction
No content moderation	`SessionProof` covers forwarding and quality metrics rather than content; under opt-in E2EE the relay is content-blind.	Content-policy mechanisms are outside the evaluated scope.
RoomCapability revocation and TURN-HMAC expiry	The controls are separate. `RoomCapability` revocation closes the signalling WebSocket after cache invalidation, but the relay data plane does not subscribe to the event. TURN HMAC credentials have no pre-expiry revocation.	Add relay-plane subscription and an explicit credential-revocation event.
Per-relay-secret leak	Rotation primitives exist, but operational coturn reload was not demonstrated. `RelaySlashed` blocks new issuance and does not rotate the secret.	Use protected key storage and evaluate operational coturn reload.
Pairing, failover, and governance	Equal scores are grouped and the first assignment vector is accepted; active membership is not filtered by recent heartbeats. Promotion is permissionless, but standby operations retain `AdminCap`. Media rejoining, recovery time, and region assignment remain open.	Bind quorum to full vectors, evict stale members, and implement timed media recovery.

Continued on the next page

Table 6.4 (continued)

Limitation	**Threat and implemented bound**	**Future-work direction**
No automatic clock-skew detector	Cross-daemon comparisons assume bounded skew. `clock_skew_warning` is a passive benchmark flag with no drift monitor. Reported latency avoids cross-host timestamp subtraction, and forensics rely on signatures rather than clock ordering.	Add automatic detection and enforce Network Time Protocol discipline.
E2EE is opt-in and fails open	SFrame is optional, transform attachment may fail open, the relay is content-blind only when the transform attaches successfully, and key distribution uses an out-of-band invite secret. The measured latency overhead is reported in §5.2.8 and summarised in §5.6.4.	Add fail-closed operation, managed keys, rekeying, and a consumer-ISP-path experiment.
No project-wide threat model	Separate encryption, threat-surface, and media-canary analyses exist, but no project-wide STRIDE analysis, attack tree, or adversarial economic model was completed.	Conduct an integrated threat analysis and evaluate explicit economic scenarios.

These limitations do not have equal significance. Proof binding and penalty custody directly constrain the central accountability claim. The prototype supports signed evidence, reward settlement, and the recording of penalty obligations followed by cooperative collection; it does not provide permissionless, end-to-end penalty enforcement. Resource democrati-

sation is supported only at the registry level; room-lifetime locking remains open. Autonomy depends on Sui and retains `AdminCap` and single-instance dependencies. Resilience evidence establishes inter-relay forwarding and on-chain promotion, but not post-promotion media continuity or mean time to recovery. Region-aware selection also remains open; the wide-area experiment supplies lower-bound latency evidence rather than validation of assignment policy.

6.4 Sui-Specific Caveats

Some properties discussed in Chapters 3–5 belong to the architecture and could, in principle, be adapted to another Move-based layer-one network. Others depend specifically on Sui's gas model, checkpoint cadence, and validator-set assumptions. The following four caveats separate these claim types.

6.4.1 Localnet Measurement and Session Trajectory

Section 5.3 reports direct `effects.gasUsed` measurements for 20 contract functions and one complete primary-and-standby ($K = 2$), four-validator trajectory on a localnet built from the pinned framework revision. The executed session contains eight proof submissions plus five fixed lifecycle calls and costs 17,134,988 MIST on the irreversible basis and 35,048,188 MIST on the net basis. This measurement does not validate the four-validator configuration: ADR-0006 specifies a four-validator, three-of-four design, whereas implemented settlement accepts two proofs and computes their arithmetic mean (§6.3).

At the labelled design assumption of USD 1.50 per SUI, the measured $K = 2$, $N = 4$ trajectory is USD 0.025702 on the irreversible basis and USD 0.052572 on the net basis. The proposal's approximately USD 0.01 figure concerns room creation only and is not used as a complete-session

acceptance threshold. Absolute mainnet cost remains unmeasured, and aggregate signatures remain the principal architectural optimisation considered in §5.3.6.

6.4.2 Localnet Chain-Observation Timing

Every chain-anchored coordination signal depends on a settlement round-trip. Section 5.2 measures a narrower pinned-localnet path from immediately before the SDK submit call to exact `RoomCreated` or `RewardsDistributed` delivery through the unchanged production event poller. At a 5 s poll interval, create has $p_{50}/p_{95}/p_{99}$ = 5003.2/5017.9/5020.4 ms and settlement has 4742.6/5120.2/5183.7 ms ($n = 30$ each). These sequential shared-state samples include polling cadence and are not consensus or mainnet finality. The architecture keeps the round trip outside steady-state forwarding, which remains a partitioning property rather than proof of the sub-200 ms target. Browser, join, revocation, promotion, and complete recovery delays remain unmeasured; reducing them could require a faster substrate or a side-channel control plane, each reopening a trust and design choice.

6.4.3 Localnet versus Mainnet Divergence

The figures in §5.3 were measured on the pinned localnet rather than mainnet. The native gas schedule, storage charges for persistent `Table` growth, and the reference gas price may all change the absolute MIST cost. The localnet ordering suggests that validator-proof scale-up dominates the evaluated optional penalty terms, but this ordering is not established for mainnet unless the relevant multipliers remain comparable. Both the absolute cost and the relative ordering therefore require a mainnet cross-check.

6.4.4 Sui's Validator Set versus DVConf's Validator Set

The term "validator" has two meanings. **Sui validators** finalise chain transactions; DVConf neither nominates nor penalises them. **DVConf validator daemons** (§4.3.4) submit dual-signed application-layer `SessionProof` artefacts. ADR-0006 records a four-validator, three-of-four design, but implemented settlement requires only two proofs and uses their arithmetic mean. The daemon's composite probe combines STUN measurements with relay HTTP counters; it is not independent participant-media verification. Chapters 3–5 use "validator" for the DVConf daemon unless "Sui validator" is stated explicitly.

The caveats split into measurement-side (the pinned-localnet $K = 2$, $N = 4$ cost trajectory, the polling-inclusive local chain observation, and mainnet divergence) and framing-side (application observation versus consensus finality and the two validator sets). Read with §6.3, they bound what the prototype establishes. A separate in-process real-mediasoup warm-pipe bench completed 30 cutovers with zero failures and measured $p_{50}/p_{95}/p_{99} = 58/74/80$ ms (§5.5.1). This is an optimistic detection-and-resume mechanism floor: it does not measure the complete sequence from relay-process termination, through on-chain promotion, to browser-rendered media recovery, so full continuity and end-to-end mean time to recovery remain open. Chapter 7 returns to TURN deployment, public-network chain and gas replication, and full join and recovery latency.

Table 6.1: Implementation status of the proposal's eight components.

#	Component	Status	Implemented boundary
1	Clients	Reduced scope	Browser implementation; IoT and mobile clients are outside the evaluated scope (§3.1.2).
2	Signalling node	Reframed	Split across services: the relay WebSocket carries runtime media signalling, while `apps/signaling` retains the earlier offer-and-answer and discovery functions (§4.3).
3	Control plane	Reduced scope	Implements quorum-based assignment (§3.7), but not the proposal's continuous real-time orchestration.
4	Load balancer	Reframed	No discrete component; control-plane daemons perform on-chain quorum-based assignment. A four-of-five strict supermajority was exercised on a single-host localnet with $N = 5$; the evaluated default uses one control-plane daemon, and multi-host operation remains open (§3.7).
5	Layer mixer	Reduced scope	The SFU is the primary mode and supports three-layer simulcast; it is content-blind only where the E2EE transform attaches successfully. The optional MCU composite is plaintext-only, rejects E2EE rooms, and was not evaluated experimentally. Estimator-driven layer switching and SVC are outside scope.
6	CDN	Out of scope	Real-time SFU delivery is evaluated; stored media distribution through a CDN is outside scope (§3.1.2).
7	Storage	Deferred	Walrus storage is deferred. `SessionProof` anchors metadata but does not provide a media archive.
8	Trust node	Reduced scope	Implemented at per-*session* granularity rather than the proposal's per-*transaction* granularity.

Table 6.2: Status of deferred and reformulated work.

Group	**Status**	**Boundary and remaining work**
IoT and mobile clients	Open	Client ports; for IoT, a minimal chain-client interface preserving signed accountability.
Walrus storage	Open	Blob upload and a chain-anchored hash.
Multi-relay delivery and orchestration	Mixed	Static $K = 2$ mesh implemented; automatic cascade-tree startup, re-parenting, and real-time orchestration open.
Stored media distribution	Reframed	Real-time SFU delivery implemented; stored playback and CDN distribution outside scope.
TURN and STUN	Partial	Issuance and rotation implemented; operational coturn provisioning and reload unverified.
Capacity, layer selection, and regional assignment	Mixed	Measured through $N \leq 15$; $N = 100$ projected. Simulcast implemented; estimator-driven selection and region-aware assignment open.
Audit and logging	Reframed	Internal analysis and per-session anchoring implemented; no independent audit conducted, and per-transaction logging not implemented.

Table 6.3: Reconciliation of earlier-reported commitments with the final state.

Reported item	**Final state**
Pion WebRTC sidecar	Superseded when mediasoup became the primary SFU.
Pre-expiry `RoomCapability` revocation	Implemented.
Secret rotation	Exists as a mechanism but lacks verified operational coturn reload.
`AdminCap` to control-plane quorum	Only capability issuance has migrated.
Three adversarial economic scenarios	Not completed.
Controlled-impairment sweep	Not performed; the wide-area campaign improves external validity but is not an equivalent experiment.
Participant sweeps at 10, 20, and 50	Not completed as specified; measurements extend through $N \leq 15$, whereas $N = 100$ remains a projection (§5.4).
Ten-connection same-source-IP limit	An anti-abuse control rather than a capacity result.
Project-wide STRIDE analysis, attack tree, and adversarial economic model	Absent; only scoped security analyses exist.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This thesis investigated how a public-chain control and economic plane can be combined with an off-chain real-time media plane. DVConf uses mediasoup for steady-state forwarding and Sui for registration, assignment, signed session evidence, reward settlement, and penalty-obligation records. A forensic command-line tool reconstructs selected emitted events, not media or consensus state. The contribution is this integrated and reproducibly evaluated boundary, not the invention of slashing, Byzantine fault tolerance, capability tokens, or selective forwarding.

Evidence against the proposal's four research gaps is deliberately differentiated.

Resource utilisation (gap 1). The relay registry permits public applications but gates admission by stake and voting; no IoT-class relay was evaluated, forwarding scaled linearly only through 15 viewers, and 100 viewers remains a projection.

Real-time performance (gap 2). 30 wide-area sessions produced a one-way p_{95} component-sum estimate of 70.9 ms. This result is a reduced-fidelity lower bound, not validation of the sub-200 ms capture-to-paint target, because it halves measured round-trip times, excludes real-camera capture, and assigns zero to the unobserved presentation term.

Incentives and quality of service (gap 3). Validators sign a composite STUN and relay-counter probe. Proof acceptance verifies an assigned validator and a registered relay, but not that the submitted relay belongs to the room. Settlement accepts two distinct proofs and takes their arithmetic mean; the designed four-validator, three-of-four rule is not enforced. Deduction is cooperative rather than automatic seizure: `RelaySlashed` records an obligation and reputation effect, and the actual deduction requires a later `pay_slash` transaction authorised by the relay-owned `StakePosition`; permissionless seizure was not demonstrated.

System integration (gap 4). The working prototype separates Sui settlement from off-chain media and measures the local SDK-call-to-production-poller boundary for room creation and settlement. Room capabilities, TURN credentials, and relay admission remain distinct enforcement surfaces; live relay admission does not enforce `RoomCapability` cryptographically.

A complete session's on-chain cost came to a few US cents at the labelled rate: the executed $K = 2$, $N = 4$ full-session trajectory of 13 transactions, measured on the pinned localnet alongside per-function gas, totalled about 0.035 SUI net (about 0.017 SUI irreversible); the exact figures appear in §5.3. Absolute mainnet cost remains unmeasured, and the proposal's preliminary USD 0.01 estimate applies only to room creation and is not treated as a complete-session acceptance threshold. Chain-side coordination became visible to off-chain services about five seconds after submission: on the same class of pinned localnet, room-creation and settlement events were delivered by the unchanged production poller about 5.0 s and 4.7 s (p_{50}) after the SDK submit call ($n = 30$ sequential observations each), a figure that includes the poller's 5 s interval and warm, non-independent chain state; the exact distributions appear in §5.2. This is not mainnet finality or user-visible join and recovery latency.

Failover was demonstrated at the mechanism level rather than as live user-visible recovery. A separate in-process bench using real mediasoup

completed 30 cutovers onto a pre-established standby media pipe (the warm-pipe mechanism) with zero failures and measured detection plus resume at $p_{95} = 74$ ms; the full distribution appears in §5.5. Warm-standby on-chain promotion was also observed, including a second sequential promotion. Live process-kill-to-browser-rendered-media continuity and full recovery time, however, were not measured.

Finally, enabling SFrame-style end-to-end encryption added a +1.86 ms mean one-way overhead on matched Azure inter-region cloud-WAN paths ($n = 30$ per arm; 95% CI $[+1.14, +2.60]$ ms), small relative to the interactive budget; the tail (p_{95}) difference was unresolved, and consumer-access-network paths remain unmeasured. The prior loopback comparison had been unresolved at $n = 12$ per arm, and the encryption is opt-in and may fail open if browser transform attachment fails.

7.2 Future Work

Future work should first close two primary evidence gaps: synchronised real-camera capture-to-paint measurement across independent access networks, and public-network replication of the pinned $K = 2$ cost and chain-observation campaigns. Failover experiments should measure process kill, detection, `RelayPromoted` visibility, transport re-establishment, browser-rendered resumed reception, and full mean time to recovery; the retained in-process warm-pipe result is only a mechanism floor. The matched encryption comparison should be extended from Azure data-centre paths to consumer access networks, and should assess rekey behaviour and fail-closed failure handling, which the executed cloud-WAN window did not measure.

Accountability work should bind each proof to the room's assigned relay, enforce the intended quorum or revise the design, remove dependence on a penalised relay voluntarily presenting its stake object, and supplement relay-reported counters with participant-observed or independently

observed media evidence (§6.3, Table 6.4). Operational and scale work should then add TURN secret rotation and expiry tests, fail-closed encryption with managed rekeying, a project-wide adversarial analysis, and only afterwards evaluate a 100-viewer session, multi-region assignment, larger relay meshes, and IoT participation as deployment claims (§6.2).

7.3 Closing Remarks

DVConf therefore demonstrates a bounded, inspectable integration of public-chain records with off-chain conferencing, not a finished decentralised service. Its explicit dispositions identify the path from prototype evidence to stronger measurement, custody, recovery, and adversarial guarantees.

References

[1] Savoir-faire Linux. “Jami: Free and universal communication platform. ”[Online]. Available: https://jami.net.

[2] Savoir-faire Linux. “OpenDHT: A lightweight C++17 distributed hash table library. ”[Online]. Available: https://github.com/savoirfairelinux/opendht.

[3] B. Allison. “Huddle01: Blockchain-based video conferencing platform built on arbitrum orbit, targets $37m node sale,” CoinDesk. [Online]. Available: https://www.coindesk.com/tech/2024/09/25/huddle01-blockchain-video-conferencing-project-that-seeks-to-outdo-zoom-targets-37m-node-sale.

[4] Huddle01. “dRTC chain: Decentralized real-time communication infrastructure,” Accessed: Jul. 10, 2026. [Online]. Available: https://huddle01.com/media-node/documentation/drtc-network/drtc-chain.

[5] Datagram Network. “Datagram: A layer-1 blockchain for real-time communication — architecture overview. ”[Online]. Available: https://doc.datagram.network/datagram-architecture.

[6] mediasoup contributors. “Mediasoup: Cutting-edge WebRTC video conferencing — documentation and source,” Accessed: Mar. 1, 2026. [Online]. Available: https://mediasoup.org.

[7] L. F. G. Sarmenta, "Sabotage-tolerance mechanisms for volunteer computing systems," in *Proc. First IEEE/ACM Int. Symp. on Cluster Computing and the Grid (CCGrid)*, Brisbane, Australia, May 2001, pp. 337–346. DOI: 10.1109/CCGRID.2001.923210.

[8] E. Buchman, "Tendermint: Byzantine fault tolerance in the age of blockchains," M.Sc. thesis, Dept. of Computer Science, University of Guelph, Guelph, ON, Canada, Jun. 2016.

[9] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. 3rd USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, New Orleans, LA, USA, Feb. 1999, pp. 173–186.

[10] J. Rosenberg, "Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal for offer/answer protocols," IETF, RFC 5245, Apr. 2010. DOI: 10.17487/RFC5245.

[11] A. Keranen, C. Holmberg, and J. Rosenberg, "Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal," IETF, RFC 8445, Jul. 2018. DOI: 10.17487/RFC8445.

[12] coturn contributors. "Coturn: Free open source implementation of TURN and STUN server," Accessed: Mar. 1, 2026. [Online]. Available: https://github.com/coturn/coturn.

[13] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," IETF, RFC 8446, Aug. 2018. DOI: 10.17487/RFC8446.

[14] E. Rescorla, "WebRTC security architecture," IETF, RFC 8827, Jan. 2021. DOI: 10.17487/RFC8827.

[15] D. McGrew and E. Rescorla, "Datagram transport layer security (DTLS) extension to establish keys for the secure real-time transport protocol (SRTP)," IETF, RFC 5764, May 2010. DOI: 10.17487/RFC5764.

[16] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman, "The secure real-time transport protocol (SRTP)," IETF, RFC 3711, Mar. 2004. DOI: 10.17487/RFC3711.

[17] International Telecommunication Union, "One-way transmission time," ITU-T, Recommendation G.114, May 2003.

[18] Mysten Labs. "Sui: Blockchain platform built on the Move programming language — documentation and Move reference," Accessed: May 1, 2026. [Online]. Available: https://docs.sui.io.

[19] I. Foster, C. Kesselman, and S. Tuecke, "The anatomy of the grid: Enabling scalable virtual organizations," *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, Aug. 2001. DOI: 10.1177/109434200101500302.

[20] R. Buyya, D. Abramson, and J. Giddy, "Economic models for resource management and scheduling in grid computing," *Concurrency and Computation: Practice and Experience*, vol. 14, no. 13–15, pp. 1507–1542, Nov. 2002. DOI: 10.1002/cpe.690.

[21] M. Armbrust et al., "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, Apr. 2010. DOI: 10.1145/1721654.1721672.

[22] Q. Zhang, L. Cheng, and R. Boutaba, "Cloud computing: State-of-the-art and research challenges," *Journal of Internet Services and Applications*, vol. 1, no. 1, pp. 7–18, May 2010. DOI: 10.1007/s13174-010-0007-6.

[23] B. P. Rimal, E. Choi, and I. Lumb, "A taxonomy and survey of cloud computing systems," in *Proc. 5th Int. Joint Conf. on INC, IMS and IDC (NCM 2009)*, Seoul, Korea, Aug. 2009, pp. 44–51. DOI: 10.1109/NCM.2009.218.

[24] LiveKit contributors. “Livekit: Open source WebRTC cloud — architecture, server, and protocol documentation,” Accessed: Mar. 1, 2026. [Online]. Available: https://livekit.io.

[25] E. Omara, J. Uberti, S. Garcia Murillo, R. Barnes, and Y. Fablet, “Secure frame (SFrame): Lightweight authenticated encryption for real-time media,” IETF, RFC 9605, Jul. 2024. DOI: 10.17487/RFC9605.

[26] K.-F. Ng, M.-Y. Ching, Y. Liu, T. Cai, L. Li, and W. Chou, “A P2P-MCU approach to multi-party video conference with WebRTC,” *International Journal of Future Computer and Communication*, vol. 3, no. 5, pp. 319–324, Oct. 2014. DOI: 10.7763/IJFCC.2014.V3.323.

[27] Huddle01. “Huddle01 dRTC unveils its L3 on Arbitrum, powered by Caldera,” Huddle01, Accessed: Jul. 10, 2026. [Online]. Available: https://huddle01.com/blog/huddle01-drtc-unveils-its-l3-on-arbitrum-powered-by-caldera.

[28] Caldera. “Huddle01 joins the caldera ecosystem by launching its Layer 3 network on Arbitrum,” Caldera, Accessed: Jul. 10, 2026. [Online]. Available: https://caldera.xyz/blog/huddle01-joins-caldera-ecosystem.

[29] Datagram Labs. “Datagram SDK documentation. ”[Online]. Available: https://docs.datagram.network.

[30] L. Lamport, “The part-time parliament,” *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, May 1998. DOI: 10.1145/279227.279229.

[31] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proc. 2014 USENIX Annual Technical Conference (USENIX ATC ’14)*, Philadelphia, PA, USA, Jun. 2014, pp. 305–319.

[32] S. Nakamoto. “Bitcoin: A peer-to-peer electronic cash system. ”[Online]. Available: https://bitcoin.org/bitcoin.pdf.

[33] S. King and S. Nadal. “Ppcoin: Peer-to-peer crypto-currency with proof-of-stake. ”[Online]. Available: https://www.peercoin.net/whitepapers/peercoin-paper.pdf.

[34] J. Kwon and E. Buchman. “Cosmos: A network of distributed ledgers. ”[Online]. Available: https://v1.cosmos.network/resources/whitepaper.

[35] Akash Network. “Akash network: A decentralized cloud computing marketplace. ”[Online]. Available: https://akash.network/whitepaper.